

Big Data

Pig e sua Aplicação em Data Warehousing

Agenda

Pig - Linguagem de Alto Nível

- Paradigma de programação declarativa.
- Operações Pig básicas.

Estratégias Eficientes para Data Warehousing

- Design de esquema de armazenamento eficiente.

Integração do Pig com o Ecossistema

- Uso conjunto com Hadoop, Hive, etc.

O que é Pig ?

- O **Apache Pig** é uma abstração sobre o MapReduce. O **Pig** é uma plataforma para análise de grandes conjuntos de dados, composta por uma linguagem de alto nível para expressar programas de análise de dados, juntamente com infraestrutura para avaliar esses programas.
- O **Apache Pig** é uma plataforma de alto nível para criar programas que são executados no Apache Hadoop.
- Para escrever programas de análise de dados, o Pig fornece uma linguagem de alto nível chamada **Pig Latin**. Essa linguagem oferece vários operadores, por meio dos quais os programadores podem desenvolver suas próprias funções para leitura, escrita e processamento de dados.

O que é Pig ?

- O Pig pode executar seus trabalhos Hadoop no **MapReduce, Apache Tez ou Apache Spark**.
- O Pig Latin abstrai a programação da ideia do MapReduce em Java para uma notação que torna a programação MapReduce de alto nível, semelhante à do SQL para sistemas de gerenciamento de banco de dados relacionais.
- O Pig Latin pode ser estendido usando **user-defined functions (UDFs)**, que o usuário pode escrever em Java, Python, JavaScript, Ruby ou Groovy e, em seguida, chamar diretamente da linguagem.

O que é Pig ?

- O Pig é geralmente utilizado com o **Hadoop**; todas as operações de manipulação de dados no Hadoop podem ser realizadas usando o Apache Pig.
- Para analisar dados usando o **Apache Pig**, os programadores precisam escrever scripts usando a linguagem Pig Latin. Todos esses scripts são internamente convertidos em tarefas de Map e Reduce. O Apache Pig possui um componente conhecido como **Pig Engine** que aceita os scripts Pig Latin como entrada e converte esses scripts em MapReduce jobs.

História

- Em 2006, o Apache Pig foi desenvolvido como um projeto de pesquisa na Yahoo, especialmente para criar e executar trabalhos MapReduce em todos os conjuntos de dados.
- Em 2007, o Apache Pig foi disponibilizado como open source através do Apache Incubator.
- Em 2008, a primeira versão do Apache Pig foi lançada.
- Em 2010, o Apache Pig se formou como um projeto de alto nível da Apache.

Por Que Precisamos do Apache Pig?

- Programadores que não têm muita experiência em Java costumavam enfrentar dificuldades ao trabalhar com o Hadoop, especialmente ao realizar tarefas de MapReduce.
- O Apache Pig é uma bênção para todos esses programadores.
- Usando o **Pig Latin**, os programadores podem realizar tarefas de MapReduce facilmente, sem precisar digitar códigos complexos em Java.

Por Que Precisamos do Apache Pig?

- O Apache Pig utiliza uma **abordagem de múltiplas consultas**, reduzindo assim o comprimento dos códigos. Por exemplo, uma operação que exigiria que você digitasse 200 linhas de código em Java pode ser facilmente feita digitando apenas 10 linhas no Apache Pig.
- Pig Latin é uma **linguagem semelhante ao SQL**, e é fácil aprender o Apache Pig quando se está familiarizado com SQL.

Por Que Precisamos do Apache Pig?

- O Apache Pig fornece muitos operadores integrados para suportar operações de dados como junções, filtros, ordenação, etc. Além disso, ele também fornece tipos de dados aninhados como tuplas, bags e maps, que estão ausentes no MapReduce.

O Apache Pig apresenta os seguintes recursos:

Conjunto Rico de Operadores: Oferece muitos operadores para realizar operações como junção, ordenação, filtro, etc.

Facilidade de Programação: Pig Latin é semelhante ao SQL, tornando fácil escrever um script Pig se você tiver conhecimento em SQL.

Oportunidades de Otimização: As tarefas no Apache Pig otimizam sua execução automaticamente, permitindo que os programadores foquem apenas na semântica da linguagem.

O Apache Pig apresenta os seguintes recursos:

Extensibilidade: Usando os operadores existentes, os usuários podem desenvolver suas próprias funções para ler, processar e escrever dados.

UDFs (Funções Definidas pelo Usuário): Pig oferece a facilidade de criar Funções Definidas pelo Usuário em outras linguagens de programação, como Python, e chamá-las ou incorporá-las em scripts Pig.

Lida com Todos os Tipos de Dados: O Apache Pig analisa todos os tipos de dados, tanto estruturais quanto não estruturados. Armazena os resultados no HDFS.

Diferenças entre o Pig e o MapReduce:

Aspecto	Apache Pig	MapReduce
Paradigma de Processamento	Linguagem de fluxo de dados	Paradigma de processamento de dados
Nível de Abstração	Linguagem de alto nível	Baixo nível e rígido
Operação de Junção	Simples e direta	Complexa e requer mais conhecimento em Java
Acessibilidade	Simples para programadores novatos com conhecimento básico em SQL	Exige familiaridade com Java
Abordagem Multi-Query	Sim, reduzindo significativamente o comprimento do código	Não possui uma abordagem similar

Diferenças entre o Pig e o SQL:

Aspecto	Apache Pig	SQL
Linguagem	Linguagem procedural Pig Latin	Linguagem declarativa SQL
Esquema dos Dados	O esquema é opcional	O esquema é obrigatório
Modelo de Dados	Modelo de dados relacional aninhado em Pig	Modelo de dados relacional plano em SQL
Oportunidades de Otimização	Oportunidades limitadas para otimização de consulta	Maior oportunidade para otimização de consulta em SQL

Diferenças entre o Pig e o Hive:

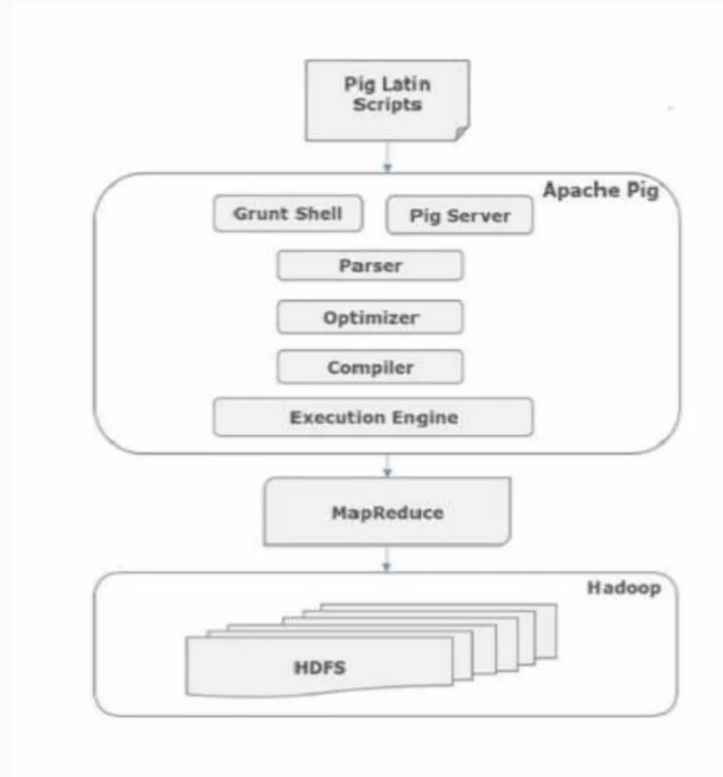
Aspecto	Apache Pig	Hive
Linguagem	Utiliza uma linguagem chamada Pig Latin.	Utiliza uma linguagem chamada HiveQL.
Origem	Criado originalmente no Yahoo.	Criado originalmente no Facebook.
Natureza da Linguagem	Linguagem de fluxo de dados.	Linguagem de processamento de consultas.
Natureza da Linguagem	Linguagem procedural que se encaixa no paradigma de pipeline.	Linguagem declarativa.
Aplicações Principais	Principalmente para dados estruturados, não estruturados e semi-estruturados.	Principalmente para dados estruturados.

Arquitetura

- É uma linguagem de processamento de dados em alto nível que fornece um conjunto rico de tipos de dados e operadores para realizar várias operações nos dados.
- Para realizar uma tarefa específica, os programadores usam o Pig, precisam escrever um script Pig usando a linguagem Pig Latin e executá-los usando qualquer um dos mecanismos de execução (Grunt Shell, UDFs, Embedded). Após a execução, esses scripts passarão por uma série de transformações aplicadas pelo Framework Pig para produzir a saída desejada.
- Internamente, o Apache Pig converte esses scripts em uma série de trabalhos MapReduce, tornando o trabalho do programador mais fácil.

Pig - Linguagem de Alto Nível

Arquitetura



Parser (Analisador)

Inicialmente, os scripts Pig são manipulados pelo Parser. Ele verifica a sintaxe do script, faz verificação de tipo e outras verificações diversas. A saída do analisador será um DAG (directed acyclic graph), que representa as instruções Pig Latin e operadores lógicos.

No DAG, os operadores lógicos do script são representados como nós e os fluxos de dados são representados como arestas.

Otimizador

O plano lógico (DAG) é enviado ao otimizador lógico, que realiza otimizações lógicas, como projeção e pushdown.

Compilador

O compilador compila o plano lógico otimizado em uma série de trabalhos MapReduce.

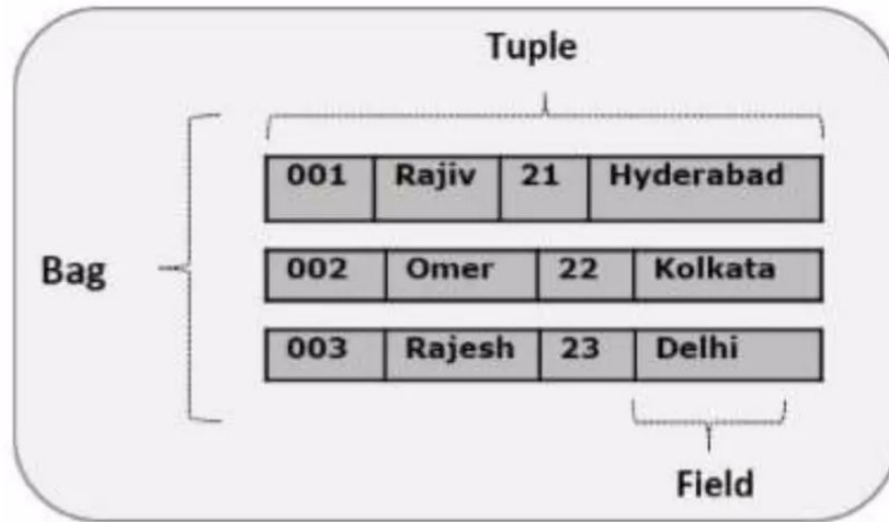
Motor de Execução

Finalmente, os trabalhos MapReduce são enviados ao Hadoop em uma ordem ordenada. Por fim, esses trabalhos MapReduce são executados no Hadoop, produzindo os resultados desejados.

Modelo de Dados Pig Latin

O modelo de dados do Pig Latin é totalmente alinhado e permite tipos de dados não atômicos complexos, como mapas e tuplas.

Design de esquema de armazenamento eficiente:



Estruturas de Dados no Pig Latin

Átomo (Atom)

Qualquer valor único no Pig Latin, independentemente do seu tipo de dado, é conhecido como um Átomo. É armazenado como string e pode ser usado como string e número. Os valores atômicos do Pig incluem int, long, float, double, chararray e bytearray. Um pedaço de dado ou um valor atômico simples é conhecido como um campo.

Exemplo: 'raja' ou '30'

Tupla (Tuple)

Um registro formado por um conjunto ordenado de campos é conhecido como uma tupla; os campos podem ser de qualquer tipo. Uma tupla é semelhante a uma linha em uma tabela de SGBD relacional.

Exemplo: (Raja, 30)

Estruturas de Dados no Pig Latin

Bag

Um saco é um conjunto não ordenado de tuplas. Em outras palavras, uma coleção de tuplas (não exclusivas) é conhecida como um saco. Cada tupla pode ter qualquer número de campos (esquema flexível). Um bag é representado por '{}'. É semelhante a uma tabela em SGBD relacional, mas, ao contrário de uma tabela em SGBD relacional, não é necessário que cada tupla contenha o mesmo número de campos ou que os campos na mesma posição (coluna) tenham o mesmo tipo.

Exemplo: {(Raja, 30), (Mohammad, 45)}

Uma bag pode ser um campo em uma relação; nesse contexto, é conhecido como bag interno.

Exemplo: {Raja, 30, {9848022338, rajawgmail.com}}

Estruturas de Dados no Pig Latin

Mapa (Map)

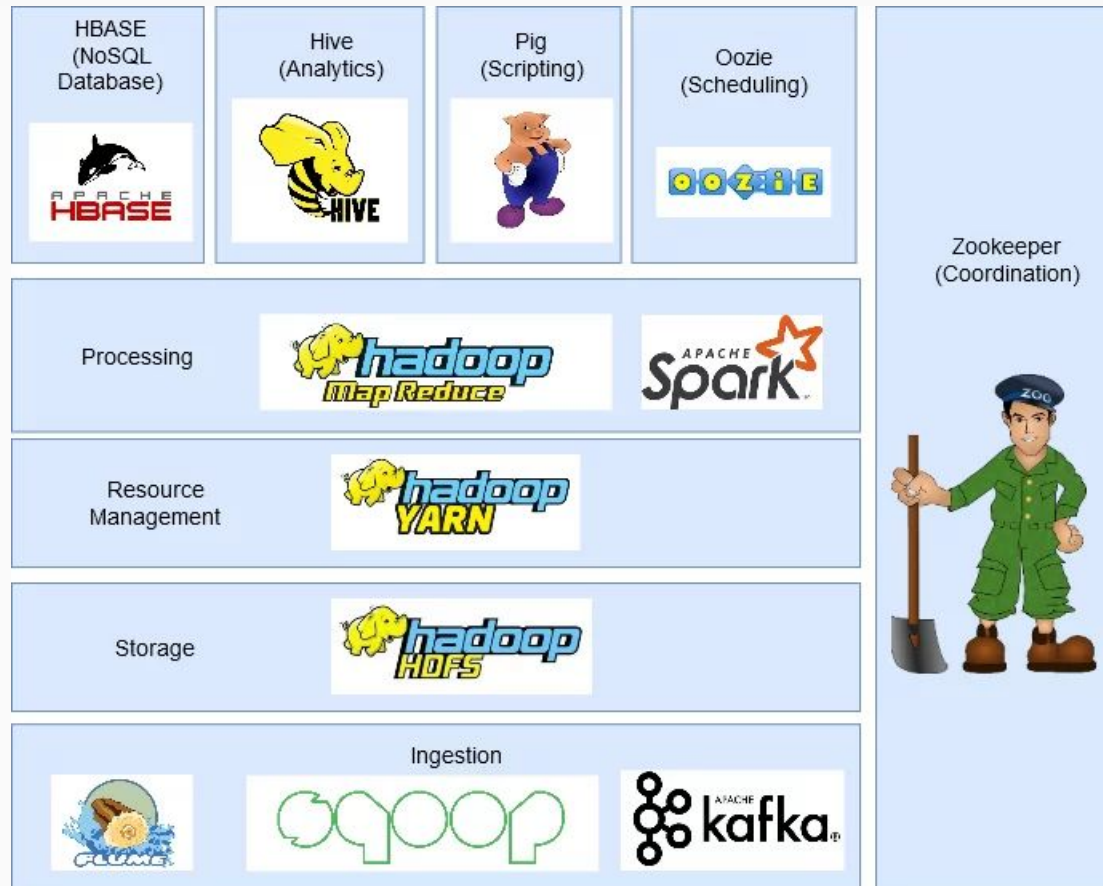
Um mapa (ou mapa de dados) é um conjunto de pares chave-valor. A chave precisa ser do tipo chararray e deve ser única. O valor pode ser de qualquer tipo. É representado por '[]'.

Exemplo: [nome#Raja, idade#30]

Relação (Relation)

Uma relação é uma bag de tuplas. As relações no Pig Latin são não ordenadas (não há garantia de que as tuplas são processadas em uma ordem específica).

Integração do Pig com o Ecosystema





FABRICIO TORQUATO

<https://www.linkedin.com/in/fabriciotorquato/>
fabricio.torquato@facens.br

Big Data

Introdução ao Spark e ETL - Parte 2

Agenda

Funcionalidades do Spark

- Outras Fontes de Dados
- Machine Learning

Outras Fontes de Dados

- Você pode conectar com outras fontes de dados
- Tratar os dados como Dataframe
- Persistir como Tabela
- Também é possível Gravar Dados em outras fontes

- **Postgresql**

ler dados

```
resumo = spark.read.format("jdbc")  
.option("url","jdbc:postgresql://localhost:5432/vendas")  
.option("dbtable","Vendas")  
.option("user","postgres")  
.option("password","123456")  
.option("driver","org.postgresql.Driver")  
.load()
```

Outras Fontes de Dados

- **Postgresql**

transformar dados

```
vendadata = resumo.select("data","total").show()
```

- **Postgresql**

gravar dados na origem

```
vendadata.write.format("jdbc")  
.option("url","jdbc:postgresql://localhost:5432/vendas")  
.option("dbtable","vendadata")  
.option("user","postgres")  
.option("password","123456")  
.option("driver","org.postgresql.Driver")  
.save()
```

- **MongoDB**

ler dados

```
resumo = spark.read.format("com.mongodb.spark.sql.DefaultSource")  
.option("url","mongodb://localhost:27017/vendas")  
.load()
```

transformar dados

```
spark.sql("SELECT * FROM vendas").show(5)
```

Uso de Spark em ambientes de machine learning

- `spark.mllib`
- `spark.ml`
- ML baseado em RDD está descontinuado
- Implementações todas em DataFrames

Variáveis Independentes

Variável Dependente



CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary	Exited
619	France	Female	42	2	0	1	1	1	10134888	1
608	Spain	Female	41	1	8380786	1	0	1	11254258	0
502	France	Female	42	8	1596608	3	1	0	11393157	1
699	France	Female	39	1	0	2	0	0	9382663	0
850	Spain	Female	43	2	12551082	1	1	1	790841	0
645	Spain	Male	44	8	11375578	2	1	0	14975671	1
822	France	Male	50	7	0	2	1	1	100628	0
376	Germany	Female	29	4	11504674	4	1	0	11934688	1
501	France	Male	44	4	14205107	2	0	1	749405	0
684	France	Male	27	2	13460388	1	1	1	7172573	0
528	France	Male	31	6	10201672	2	0	0	8018112	0
497	Spain	Male	24	3	0	2	1	0	7639001	0
476	France	Female	34	10	0	2	1	0	2626098	0
549	France	Female	25	5	0	2	0	0	19085779	0
635	Spain	Female	35	7	0	2	1	1	6595165	0
616	Germany	Male	45	3	14312941	2	0	1	6432726	0
653	Germany	Male	58	1	13260288	1	1	0	509767	1
549	Spain	Female	24	9	0	2	1	1	1440641	0
587	Spain	Male	45	6	0	1	0	0	15868481	0
726	France	Female	24	6	0	2	1	1	5472403	0
732	France	Male	41	8	0	2	1	1	17088617	0
636	Spain	Female	32	8	0	2	1	0	13855546	0

- Normalmente todas as variáveis independentes devem compor uma mesma coluna
- Cria-se um vetor único, que é adicionado em nova coluna no DataFrame

One Hot Encoding

- Se o atributo tiver muitos valores, muitas colunas serão criadas
- Spark permite o uso de matriz esparsa
- Muitos valores zero que não são registrados

Pipelines

- Transformer: Transforma um DF em outro DF
- Estimator: Fit em DF para produzir um Transformer
- Pipeline: conecta Transformers e Estimators para Produzir modelo
- Parâmetros: Transformers e Estimators compartilham uma Api para definir parâmetros

Reutilização de Dataframe

- Cache – padrão em memória
- Persist – definido pelo usuário

StorageLevel

- MEMORY_ONLY: Padrão para RDD, porém se não caber na memória será reprocessado a cada consulta
- MEMORY_AND_DISK: Padrão para DataFrame. Armazena as partições que não cabem em memória em disco

Cache e Persist

DISK_ONLY

DISK_ONLY_2

DISK_ONLY_3

MEMORY_AND_DISK

MEMORY_AND_DISK_2

MEMORY_AND_DISK_SER

MEMORY_AND_DISK_SER_2

MEMORY_ONLY

MEMORY_ONLY_2

MEMORY_ONLY_SER

MEMORY_ONLY_SER_2

OFF_HEAP



FABRICIO TORQUATO

<https://www.linkedin.com/in/fabriciotorquato/>
fabricio.torquato@facens.br

Big Data

Introdução ao Spark e ETL - Parte 1

Agenda

Fundamentos Avançados do Apache Spark

- Arquitetura Spark.

Ecossistema Spark - Visão Geral

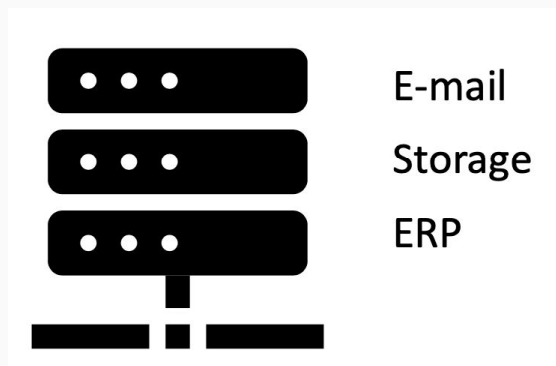
- Spark SQL, Streaming, MLlib, etc.
- Integração com outras tecnologias.

O Que é Spark ?

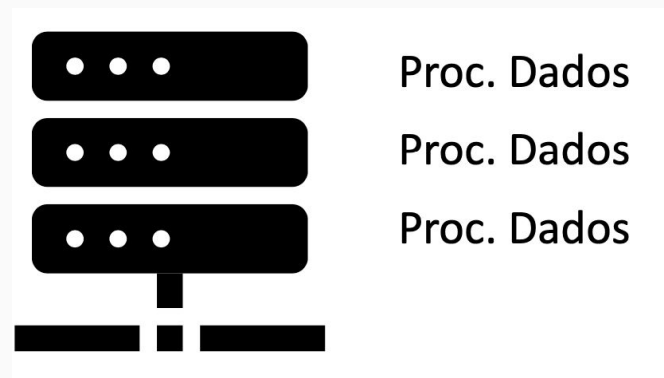
- Ferramenta de Processamento de Dados (Não é Data Storage)
- Distribuído em um Cluster
- Em memória
- Veloz
- Escalável
- Dados em HDFS ou Cloud
- Particionamento

Cluster

Rede de Computadores

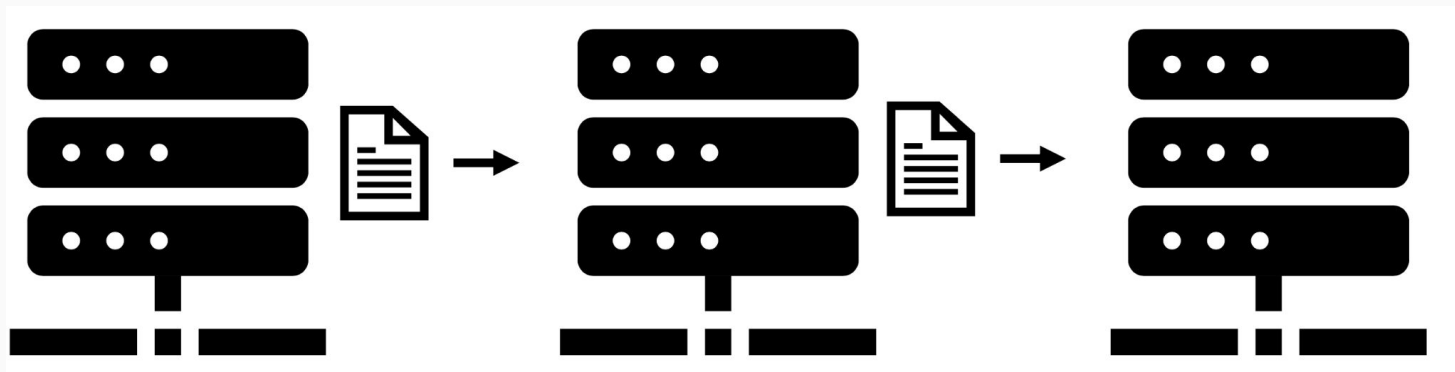


Rede de Computadores - Cluster

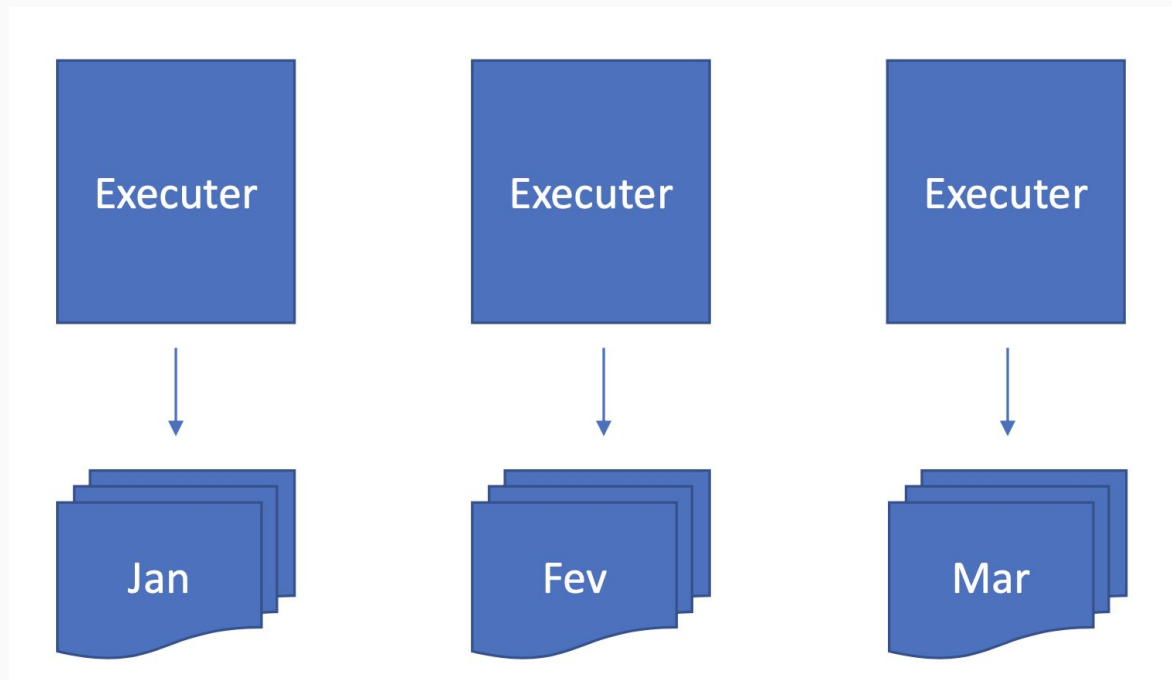


Replicação e Tolerância a Falha

Dados são copiados entre os nós do cluster. Isso traz o benefício de, entre outras coisas, tolerância a falhas



Particionamento



Spark VS Python, R ou Banco de Dados

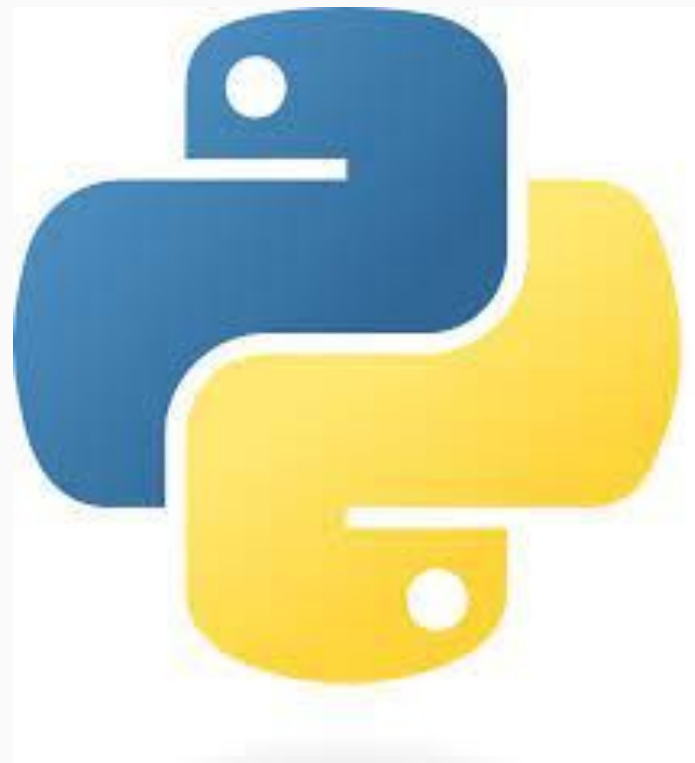
- Você precisa Processar dados!
- Custo computacional: CPU, Memória, Rede etc.
- Spark tem arquitetura voltada a processar dados!
 - Melhor performance, porém:
 - Não substitui Python
 - Não substitui SQL ou um SGBDR

Linguagens

- Scala
- Python
- Java
- R
- SQL

Linguagens

- Scala
- Python
- Java
- R
- SQL



História

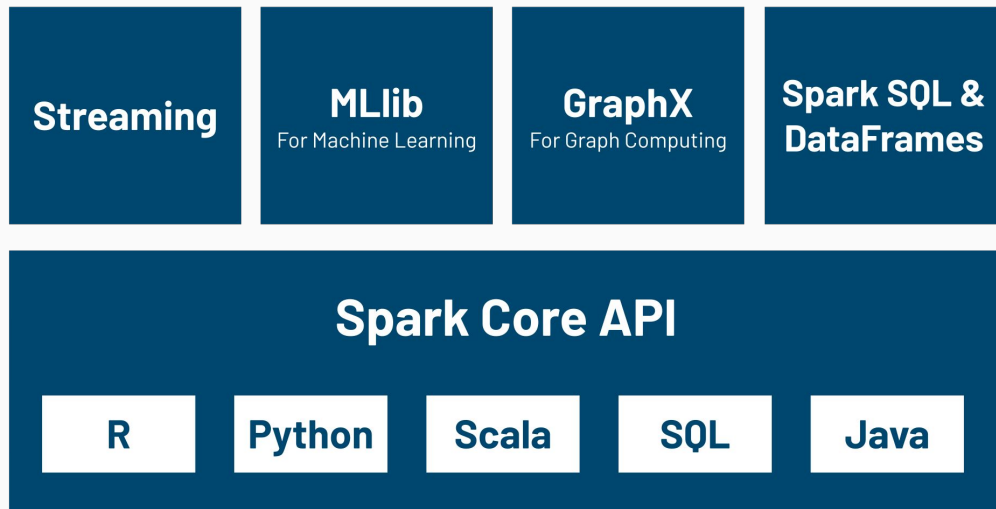
- Google File System (GFS), MapReduce (MR) e Bigtable
- Resultou em Hadoop, MapReduce, HDFS e Yarn
 - Complexo
 - Requer conhecimento em Java
 - Modelo em Batch em tarefas Mapeamento e Redução
- Solução
 - Ex: Hive criado pelo Facebook: DW SQL sobre HDFS

História

- Spark
 - Universidade da Califórnia iniciou o projeto Spark em 2009;
 - Versão 1.0 lançada em Maio de 2014 pela Fundação Apache;

Componentes

- Machine Learning(Mlib)
- SQL (Spark SQL)
- Processamento em Streaming
- Processamento de Grafos (GraphX)



Spark SQL

- Permite ler dados tabulares de várias fontes (CSV, Json, Parquet, ORC etc)
- Pode usar sintaxe SQL



Streaming: Spark Structured Streaming

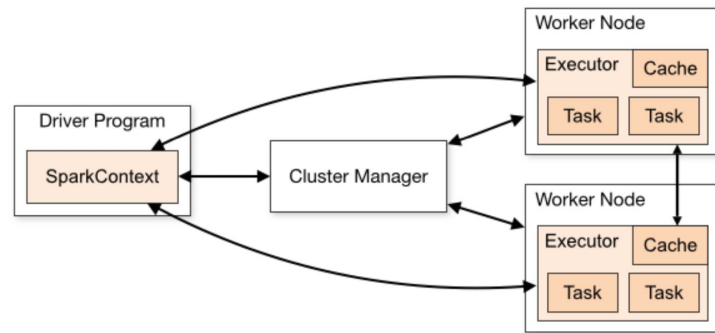
- Dados estruturados
- Novos registros adicionados ao final da tabela

Grafos acíclicos dirigidos

- Spark Constrói Gráficos Acíclicos Dirigidos
 - Acíclicos: não tem ciclo
 - Dirigidos: tem uma direção

Arquitetura Spark:

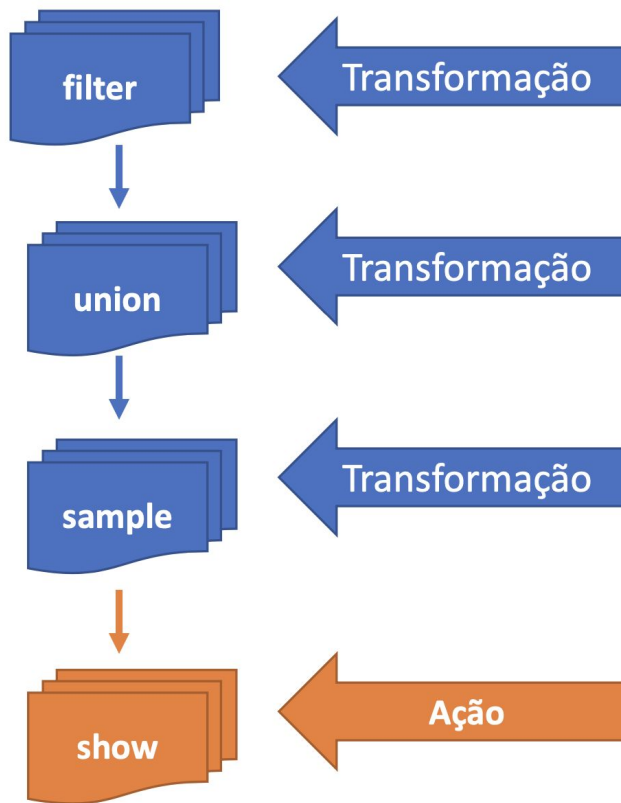
- **Driver:** Inicializa SparkSession, solicita recursos computacionais do Cluster Manager, transforma as operações em DAGs, distribui estas pelos executers
- **Manager:** Gerencia os recursos do cluster. Quatro possíveis: built-in standalone, YARN, Mesos e Kubernetes
- **Executer:** roda em cada nó do cluster executando as tarefas



Transformações e Ações

- Um dataframe é imutável: traz tolerância a falha
- Uma transformação gera um novo dataframe.
- O processamento de transformação de fato só ocorre quando há uma Ação:
Lazy Evaluation

Lazy Evaluation



Transformações e Ações

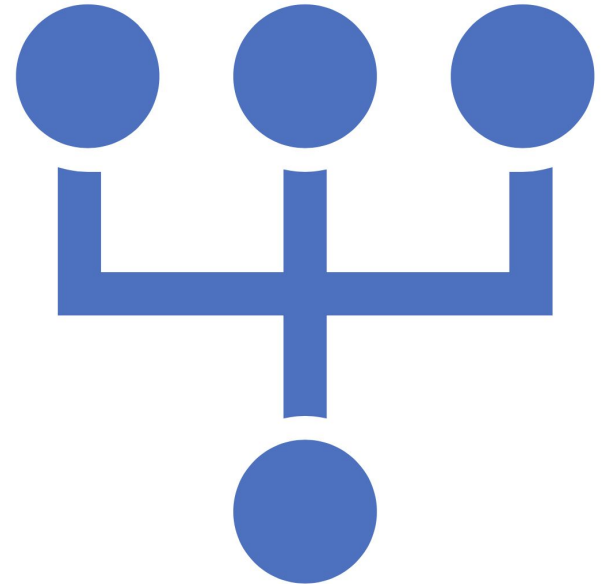
Transformações	Ações
map	
filter	reduce
flatMap	
mapPartitions	collect
mapPartitionsWithIndex	
sample	count
union	first
intersection	take
distinct	
groupByKey	takeSample
reduceByKey	takeOrdered
aggregateByKey	
sortByKey	saveAsTextFile
join	saveAsSequenceFile
cogroup	
cartesian	saveAsObjectFile
pipe	
coalesce	countByKey
repartition	
repartitionAndSortWithinPartitions	foreach

Transformações: Narrow e Wide

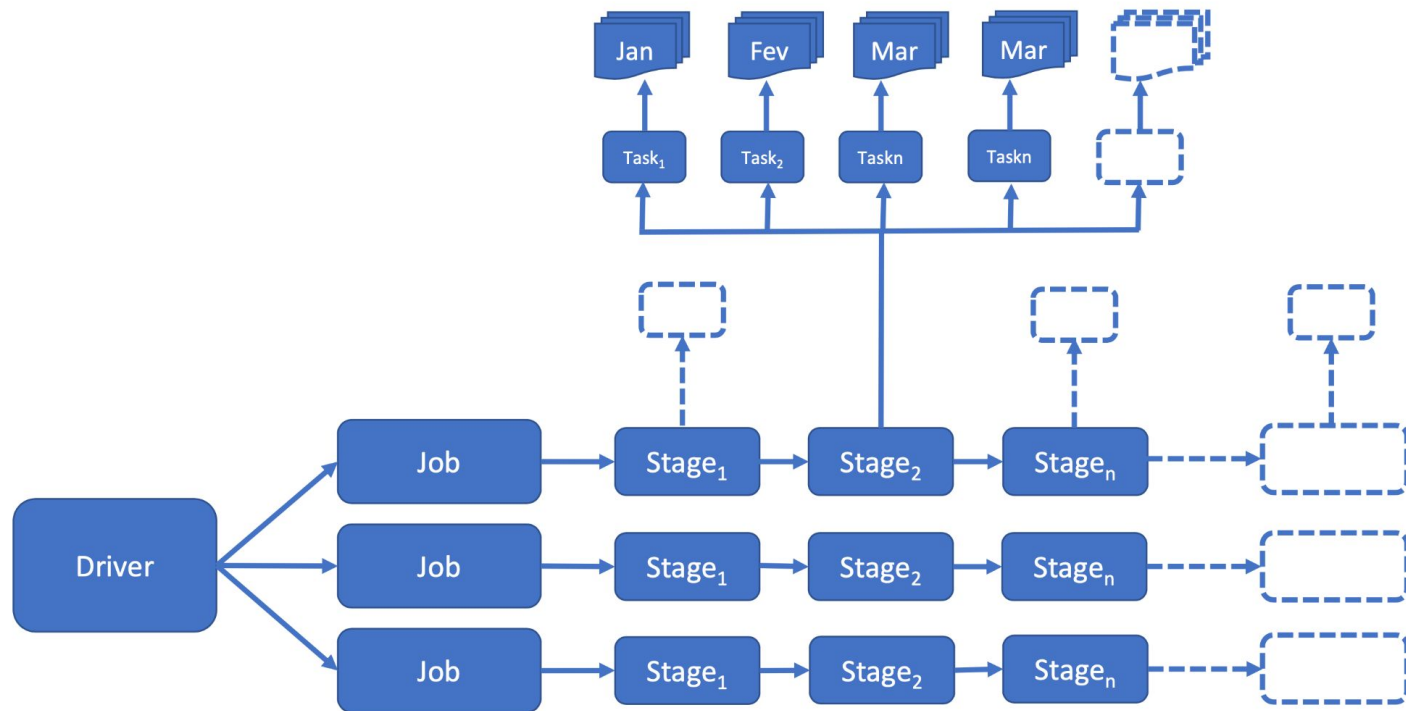
- Os dados necessários estão em uma mesma partição
- Os dados necessários estão em mais de uma partição

Componentes

- Job: Tarefa
- Stage: Divisão do Job
- Task: Menor unidade de trabalho. Uma por núcleo e por partição



Fundamentos Avançados do Apache Spark



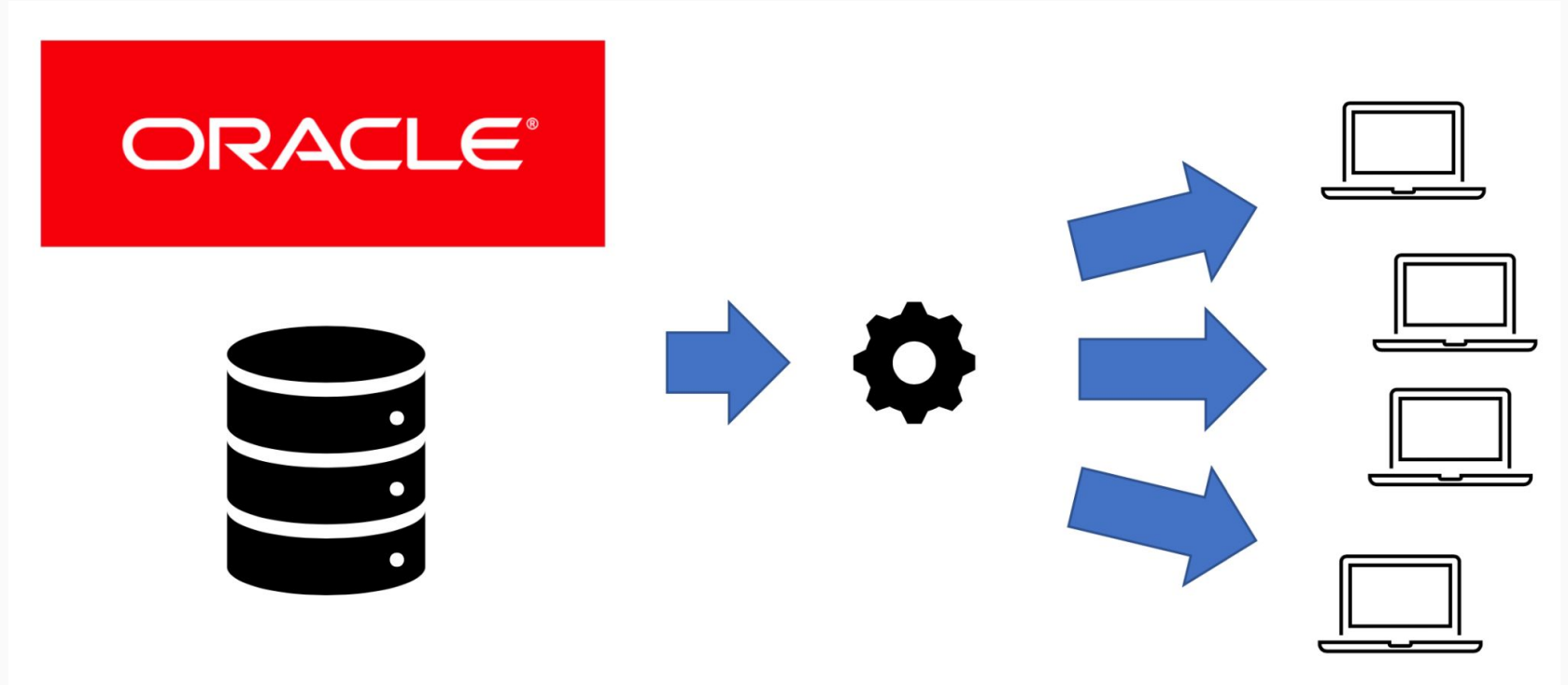
SparkContext / Session

Conexão transparente com o Cluster

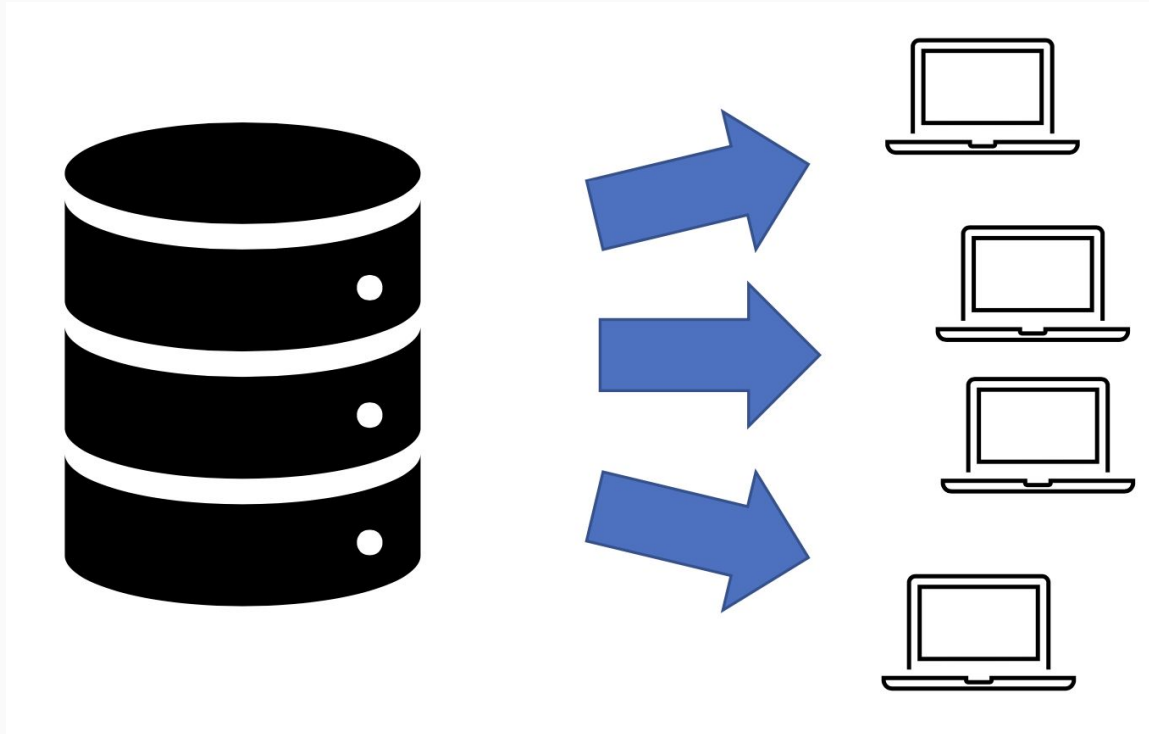
- Você pode rodar script Spark no shell (pyspark)
- O Spark cria uma sessão automaticamente chamada spark
- Se criar uma aplicação spark, você precisa criar, por exemplo:

```
spark = SparkSession.builder.appName("Meuapp").getOrCreate()
```

Formatos Proprietários



Formatos Abertos



Formatos para Big Data

- Armazéns de dados modernos tendem a armazenar dados em formatos “desacoplados” de ferramentas e abertos
- Formatos binários, compactados
- Suportam Schema
- Podem ser particionados entre discos:
 - Redundância
 - Paralelismo
- Parquet – Colunar, padrão do Spark
- ORC – Colunar, padrão do Hive



Formatos para Big Data

- Em geral ORC é mais eficiente na criação (escrita) e na compressão
- Parquet tem melhor performance na consulta (leitura)





FABRICIO TORQUATO

<https://www.linkedin.com/in/fabriciotorquato/>
fabricio.torquato@facens.br

Big Data

Apache Hive e seu Papel em Soluções de Dados

Agenda

Internos do Apache Hive

- Estrutura e componentes do Hive.
- Metastore e otimizações.
- Boas práticas

O Papel Crucial do Hive em Big Data

- Consultas SQL em grandes conjuntos de dados.

Desenvolvimentos Recentes e Casos de Uso

- Novidades em versões recentes do Hive.

O que é Hive ?

- Desenvolvido inicialmente pelo **Facebook** em 2007;
- **Objetivo:** Ajudar a empresa **gerir** melhor as **massas** de **dados** geradas naquele momento;
- Em 2007 Facebook já tinha **15TB** de **dados**;
- **Pouco tempo** depois o volume **aumentou** para **700TB**.

O que é Hive ?

- **Data Warehouse** do Facebook **demorava** demais para **processar** consultas diárias.
- Mudança do Data Warehouse para o **Hadoop**;
- **Dificuldade** de se criar jobs **MapReduce** e **alto consumo** de **tempo** dos seu funcionários.
- **Criação** do **Hive** - ferramenta com conceitos similares dos bancos de dados tradicionais

O que é Hive ?

- É um Data Warehouse **escalável, tolerante a falhas** que construído no ecossistema **Hadoop**.
- Suporta **grandes massas de dados**.
- Sistema para **processamento em batch**.
- Possui uma linguagem de consulta similar ao **Structure Query Language (SQL)** conhecida como **Hive Query Language (HiveQL)**

Quando utilizar o Hive ?

- Quando já se tem **conhecimento** de **SQL**
- **Compatível** com **outras ferramentas** do ecossistema **Hadoop** - (outras ferramentas do Hadoop **permitem** usar o **Hive** como **forma** de **armazenamento**)
- **Dados estruturados**;
- **Processamento** em **batch** (Processamento de grandes massas de dados)

Hive X Banco de dados relacionais

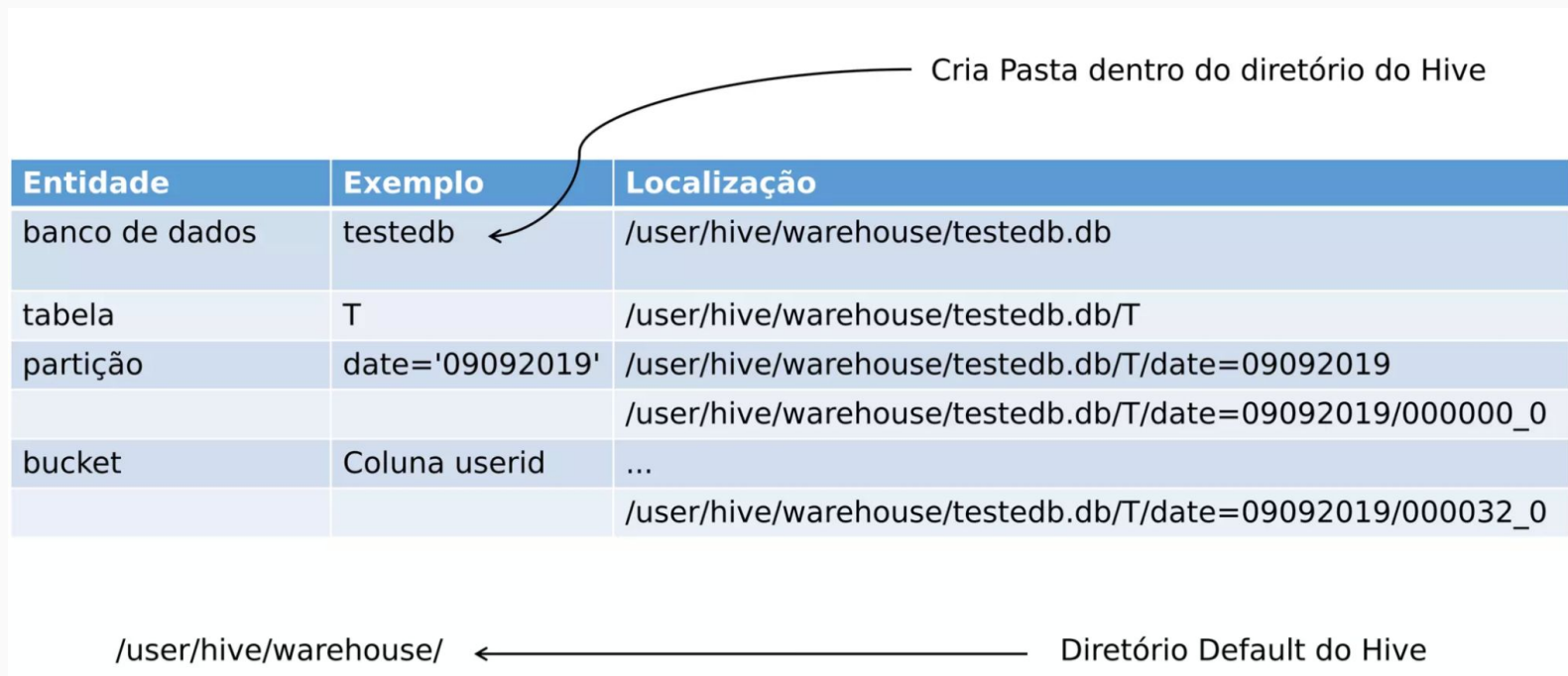
Hive	Bancos relacionais
Similar ao SQL	SQL
Análise	OLTP Sistemas Transacionais (e.g., ERP's) e Análise (e.g., Data Warehouses)
Processamento em batch	Processamento em batch e tempo real
Projetado para o processamento distribuído	Não projetado para processamento distribuído
Alta performance em computadores de baixo custo (Hadoop)	Alta performance em computadores proprietários e de alto custo
Baixo custo de armazenamento	Armazenamento caro e limitado comparado ao HDFS (Sistema de armazenamento do Hadoop)

Organização dos dados

- **Banco de dados:** É um **namespace** que **separa tabelas** e outras unidades;
- **Tabela:** Unidades homogêneas que possuem **esquema**;
- **Partição:** Uma **coluna “virtual”** que define o armazenamento dos dados no **HDFS**. Cada tabela pode ter uma ou mais partições;
- **Bucket:** Em cada partição, os dados podem ser divididos em buckets baseado no hash do valor de uma coluna;
- Essas **divisões** tanto das **partições** quanto em **buckets** permitem que o sistema **restringa o acesso** a uma **grande quantidade de dados** durante o **processamento** de uma consulta **resultando em melhor performance**.

Organização física dos dados

Entidade	Exemplo	Localização
banco de dados	testedb	/user/hive/warehouse/testedb.db
tabela	T	/user/hive/warehouse/testedb.db/T
partição	date='09092019'	/user/hive/warehouse/testedb.db/T/date=09092019
		/user/hive/warehouse/testedb.db/T/date=09092019/000000_0
bucket	Coluna userid	...
		/user/hive/warehouse/testedb.db/T/date=09092019/000032_0



/user/hive/warehouse/ ← Diretório Default do Hive

Formatos de armazenamento

- **Text File:** é o formato padrão no Hive onde cada registro representa uma linha. É vantajoso quando deseja-se compartilhar os dados com outras ferramentas de edição manual dos dados. No entanto, é menos eficiente que os formatos binários.
- **Sequence File:** Hadoop Sequence File consiste de pares chave e valor. Armazena um registro como um valor binário único. É vantajoso para compartilhar os dados com outras ferramentas do ecossistema Hadoop.

Formatos de armazenamento

- **RC File (Record Columnar File)**
- **ORC File (Optimized Row Columnar File) Substituto do formato RC File.**

O objetivo dos RC Files é permitir (1) rapidez na carga dos dados, (2) rapidez no processamento das consultas, (3) eficiência no armazenamento e (4) adaptabilidade aos padrões dinâmicos de acesso aos dados. O formato RC File armazena os dados horizontalmente em grupos de colunas.

Formatos de armazenamento

Parquet: Armazena dados binários de uma forma orientada por coluna, onde os valores de cada coluna são organizados de modo que todos estejam adjacentes, permitindo uma melhor compactação. É bom para consultas que leem colunas específicas de uma grande tabela (com muitas colunas), uma vez que somente as colunas necessárias são lidas e assim o I/O é minimizado.

Formatos de armazenamento

- O formato do arquivo **Parquet** incorpora vários recursos que o tornam altamente adequado para operações de **data warehouse**;
- **Layout de armazenamento em coluna:** uma consulta pode examinar e executar cálculos em todos os valores de uma coluna enquanto lê apenas uma pequena fração dos dados de um arquivo ou tabela de dados.
- **Opções flexíveis de compressão:** os dados podem ser compactados com alguns dos vários codecs e diferentes arquivos de dados podem ser compactados de forma diferente. A compressão é transparente para aplicativos que lêem os arquivos de dados.
- **Snappy:** é uma biblioteca de compressão / descompressão. Não visa a compressão máxima, nem a compatibilidade com qualquer outra biblioteca de compressão; Em vez disso, visa velocidades muito altas e compactação razoável.

External / Internal

External

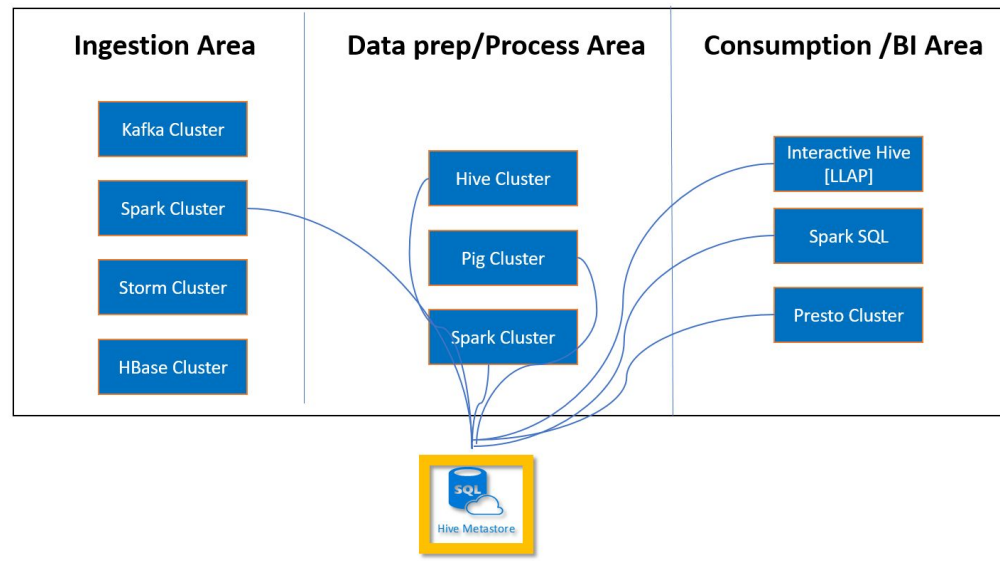
- Hive não controla o arquivo
- Arquivo pode ser utilizado por outro recurso
- Tipo recomendado
- Se excluir a tabela, apenas metadados são excluídos (Arquivo permanece)

Internal (Managed)

- Hive controla
- Acesso exclusivo pelo Hive
- Excluir tabela exclui arquivo físico

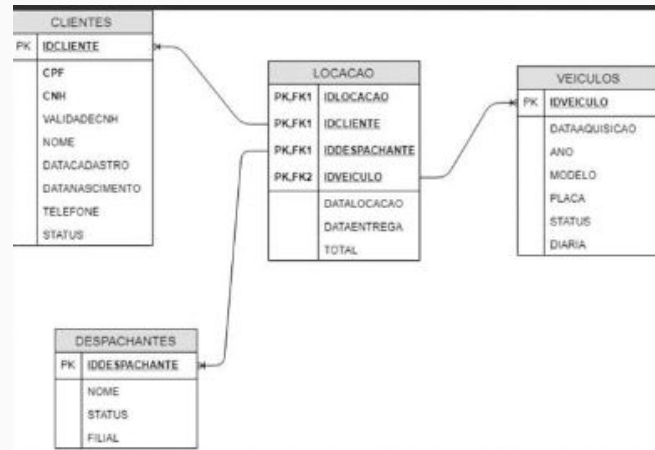
Metastore e otimizações:

O Hive armazena seus metadados de banco de dados e tabela em um metastore, que é um banco de dados ou armazenamento suportado por arquivos que permite fácil abstração e descoberta de dados.



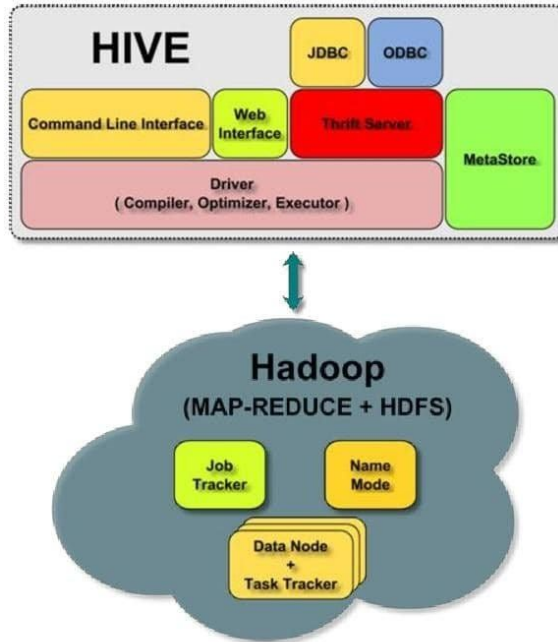
Desnormalização

A normalização é um processo padrão usado para modelar tabelas de dados lidando com redundâncias e outras anomalias. Junções são operações caras e complicadas e geram problemas de desempenho. É uma boa idéia evitar estruturas de tabelas altamente normalizadas, pois exigem a junção de consultas.



cli.nome	des.nome	veic.modelo	loc.datalocacao	loc.total
Carmina Verissimo	Graça Ornellas	Infiniti Q50S 3.7 Sedan	2019-02-20	1996.0
Clóvis Carrasco	Graça Ornellas	BMW 750i xDrive	2019-06-30	1843.0
Rebeca Torcuato	Graça Ornellas	BMW 750i xDrive	2019-03-09	2016.0
Xisto Mendoça	Emídio Dornelles	Volvo XC90 T8 Hybrid	2019-04-19	1857.0
Hipólito Granja	Viviana Sequeira	BMW 750i xDrive	2019-02-24	2049.0
Márcio Taveira	Graça Ornellas	BMW 750i xDrive	2019-05-15	1986.0

HIVE Architecture



Consultas SQL em grandes conjuntos de dados

Tipos de dados

Tipo de dado homogêneo		T i p o d e d a d o h e t e r o g ê n e o	
tinyint 1 byte	timestamp	ARRAY	ARRAY<TipoDoDado>
smallint 2 bytes	date - YYYY-MM-DD	MAP	MAP<NomeDaChave, Valor>
int 4 bytes	string	STRUCT	STRUCT<NomeDoCampo : TipoDoDado, ...>
bigint 8 bytes	varchar	UNION	UNIONTYPE<TipoDoDado, TipoDoDado, ...>
float 4 bytes	char		
float 8 bytes	boolean		
decimal - precisão 38 dígitos	binary		

Operadores relacionais

Operador	Descrição
$A = B$ $A == B$	Retorna true se a expressão A for igual a expressão B e false caso contrário.
$A <=> B$	Retorna o mesmo resultado do operador = para A e B não nulos. Retorna true se ambos forem NULL e false se apenas um deles forem NULL.
$A <> B$ $A != B$	Retorna NULL se A ou B forem nulos. Retorna true se a expressão A não for igual a expressão B e false caso sejam iguais
$A < B$	Retorna NULL se A ou B forem nulos
$A <= B$	
$A > B$	
$A >= B$	

Operadores relacionais

Operador	Descrição
A [NOT] BETWEEN B AND C	Retorna true se A for maior ou igual a B e A for menor ou igual a C Isso pode ser invertido através da palavra NOT.
A IS NULL	Retorna true se a expressão A for nula
A IS NOT NULL	Retorna NULL se A ou B forem nulos. Retorna true se a expressão A não for igual a expressão B e false caso sejam iguais
A [NOT] LIKE B	Exemplo: 'foobar' LIKE 'foo' = false 'foobar' LIKE 'foo__' = true 'foobar' LIKE 'foo%' = true
A RLIKE B	Exemplo: 'foobar' RLIKE 'foo' ⇒ true
A REGEXP B	O mesmo que RLIKE

Consultas SQL em grandes conjuntos de dados

Funções pré-existentes

Função	Retorno	Descrição
round(double A)	bigint	Valor arredondado
floor(double A)	bigint	maior inteiro tal que seja menor ou igual ao parâmetro
ceiling(double A)	bigint	menor inteiro tal que seja maior ou igual ao parâmetro
substr(string A, int start)	string	Substring de A com início no carácter da posição start até o final da string A
upper(string A)	string	Conversão de todos os caracteres de A para letras maiúsculas
length(string S)	int	Tamanho da string
year(string timestamp)	int	Apenas o ano da string timestamp
get_json_object(string jsonString, string path)	string	Extraí de uma string JSON um objeto JSON baseado no caminho especificado. Exemplo: get_json_object(src_json.json, '\$.store.fruit[0]')

Funções pré-existentes

Função	Retorno	Descrição
count (*)	bigint	Conta o número total de linhas retornadas, incluindo linhas que possuem valores NULL.
sum(coluna)	double	Retorna a soma dos valores na coluna especificada.
avg(coluna)	double	Retorna a média dos elementos na coluna especificada.
min(coluna) max(coluna)	double	Retorna o menor ou maior valor encontrado na coluna especificada.
explode(array)	N linhas	Retorna zero ou mais linhas onde cada elemento do ARRAY se tornará uma linha distinta.
explode(map)	N linhas	Retorna zero ou mais linhas onde cada chave e valor do MAP se tornará uma linha distinta com duas novas colunas, uma para a chave e outra para o valor.

HQL

- Alguns comandos de ordenação
- ORDER BY
- DISTRIBUTED BY (é equivalente ao GROUP BY do SQL)
- CLUSTER BY (é equivalente a SORT BY do SQL)

Lendo Dados

```
select * from veiculos;
```

```
select modelo from veiculos;
```

```
select modelo from veiculos where status <> "Disponivel";
```

Ingestão de Dados

Carregando dados do HDFS

```
LOAD DATA INPATH '/user/hive/locacao/clientes.csv'  
INTO TABLE CLIENTES;
```

Carregando dados do sistema de arquivos local

```
LOAD DATA LOCAL INPATH '/user/hive/downloads/clientes.csv'  
INTO TABLE CLIENTES;
```

Salvando Dados

Salvando no HDFS

```
insert overwrite directory '/user/hive/locacao/clientes.csv' select * from locacao;
```

Salvando no sistema de arquivos local

```
insert overwrite local directory '/user/hive/locacao/clientes.csv' row format  
delimited fields terminated by ',' select * from locacao;
```

Transações

- Dados distribuídos
- Atualização significa atualizar os dados em todos os nós
- Um DW pode não precisar de update e delete
- Por padrão o Hive não suporta update e delete
- Pode ser configurado (não recomendado)

- **Arquivos ORC**
- **Partition**
- **Bucket**
- **Vetorização**
- **Cost Based Optimization**
- **Spark**

- Reduz o uso de CPU
- Processa blocos de 1024 linhas por vez
- Suporta apenas o formato ORC

- Cost-Based Optimization
- Coleta e usa estatísticas para melhorar a performance de consultas
- Tenta otimizar o plano de execução de consultas
- Trade off: overhead pela coleta de dados

Execution Engine

- Map Reduce (padrão)
- Tez
- Spark (chega a ser 3X mais rápido)

- Em tempo real / Iterativo
- Compatível com MapReduce
- Compatível com Hadoop
- Em memória / disco / híbrido
- Em cluster

Desenvolvimentos Recentes

O Apache Hive, uma ferramenta essencial no ecossistema Hadoop, tem evoluído constantemente para atender às crescentes demandas do processamento de dados distribuído. Nas versões mais recentes, várias melhorias e novidades foram introduzidas, consolidando o Hive como uma opção robusta para análise de dados em ambientes big data.

Desenvolvimentos Recentes

Suporte a ACID Transactions:

Uma das adições significativas nas versões recentes foi o suporte a transações ACID (Atomicity, Consistency, Isolation, Durability), proporcionando maior confiabilidade em operações transacionais complexas.

Aprimoramentos em Segurança:

Texto: A segurança também foi aprimorada, com atualizações nos recursos de controle de acesso e uma maior integração com sistemas de autenticação, incluindo Kerberos.

Ferramentas e Interface do Usuário:

Houve melhorias na interface do usuário do Hive, bem como uma integração mais estreita com ferramentas de desenvolvimento, facilitando o uso e a administração.

Monitoramento e Diagnóstico:

Recursos adicionais foram incorporados para facilitar o monitoramento do desempenho do Hive, oferecendo acesso a métricas e diagnósticos importantes.



FABRICIO TORQUATO

<https://www.linkedin.com/in/fabriciotorquato/>
fabricio.torquato@facens.br

Big Data

Processamento Distribuído com Hadoop e Técnica MapReduce - Parte 2

Agenda

Técnica MapReduce - Detalhes e Aplicações

- Fluxo de trabalho MapReduce.
- Exemplos práticos de implementação.

Tendências Atuais em Processamento Distribuído

- Frameworks além do Hadoop
- Processamento de dados em tempo real.

O que é MapReduce ?

- Principal tarefa: processamento paralelo;
- Simplifica o processamento de dados em grande escala;
- Pode ser utilizado tanto sobre sistemas de arquivos (dados não estruturado) quanto sobre banco de dados (dados estruturados);
- Processamento composto de duas fases:
 - Map (mapear)
 - Reduce (reduzir, simplificar)

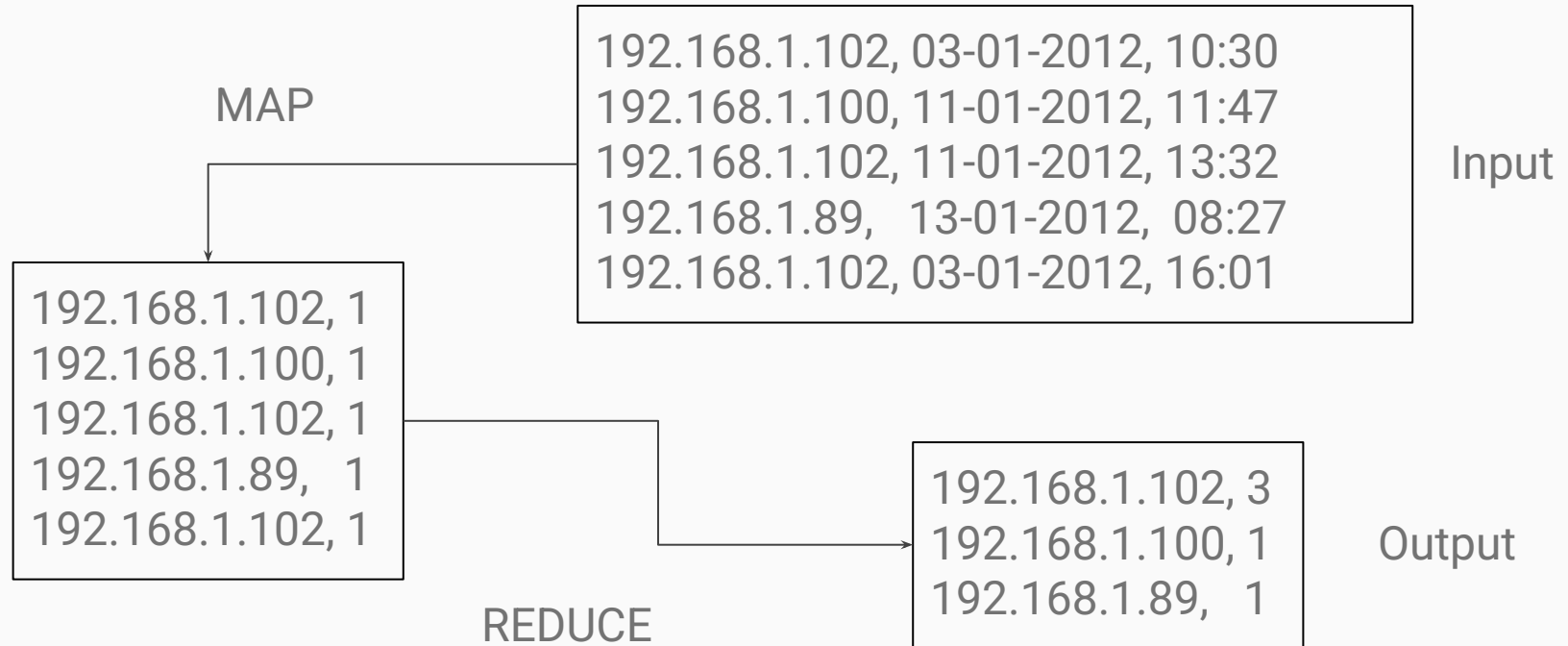
A fase MAP

- Master recebe a entrada e divide em problemas menores;
- Cada “subproblema” é então distribuído para algum worker;
- Os workers trabalham no subproblema, gravam seu resultado no disco local e depois enviam o resultado através de rede para o master;
- A saída é composta por pares de informação formados por chave-valor;

A fase REDUCE

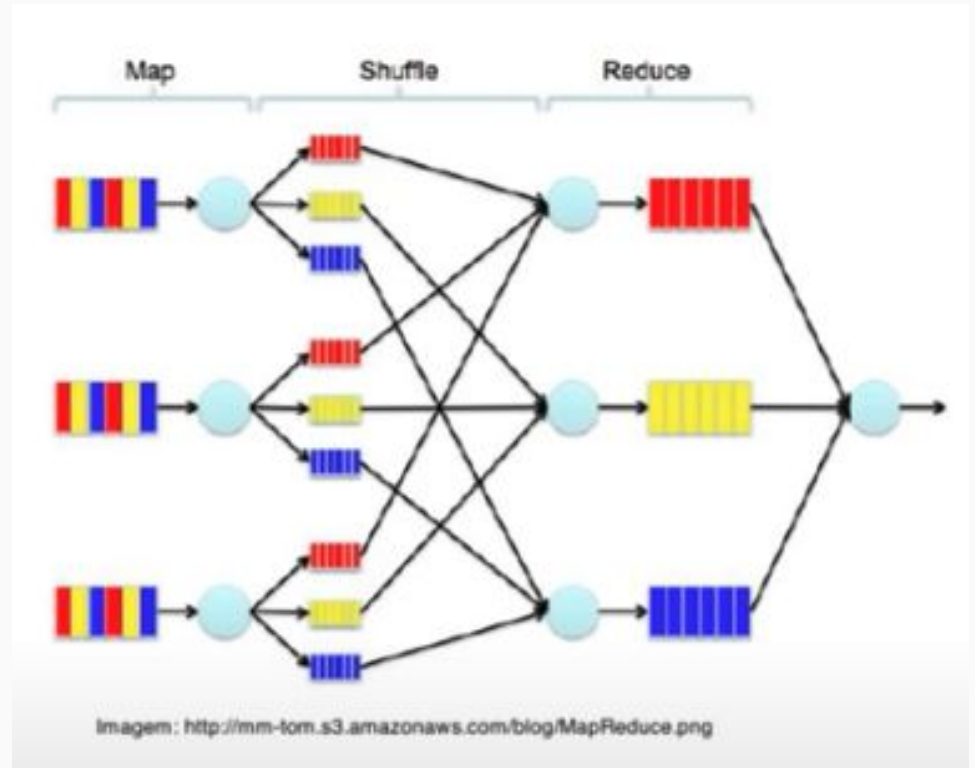
- O master recebe a resposta de todos os nós participantes do processamento;
- As respostas são, então, combinadas de alguma forma para que se obtenha a resposta do problema original;

Técnica MapReduce



Técnica MapReduce

1. Input
2. Map
3. Shuffle
4. Reduce
5. Output

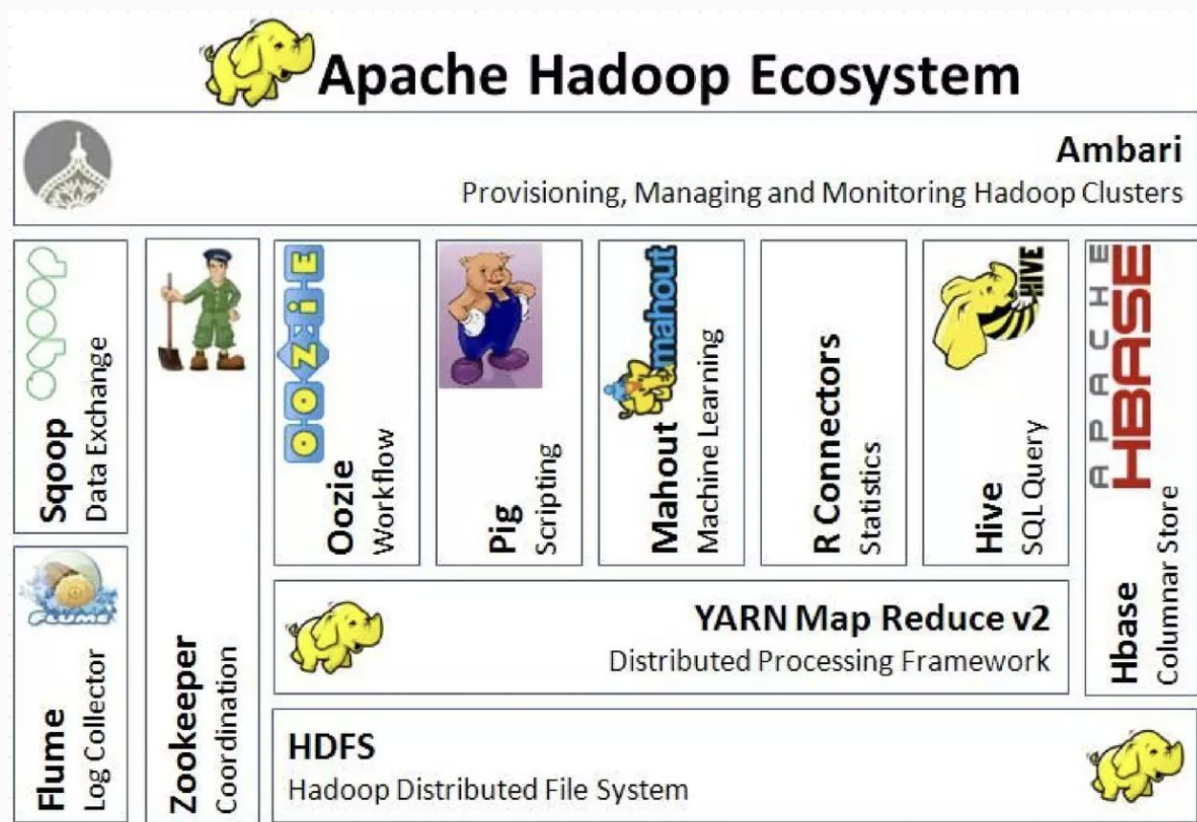


Três possibilidades de comportamento do Hadoop:

- **Local:** É executado em apenas uma JVM, sendo a configuração adequada em tempo de desenvolvimento;
- **Pseudo-distribuído:** Serviços background do Hadoop executam localmente, simulando um cluster; Também chamado Single-Node Cluster, no qual cada Hadoop daemon é rodado em um processo java separado;
- **Totalmente distribuído:** Serviços background do Hadoop executam num cluster real;

Possuem o Hadoop como mecanismo central:

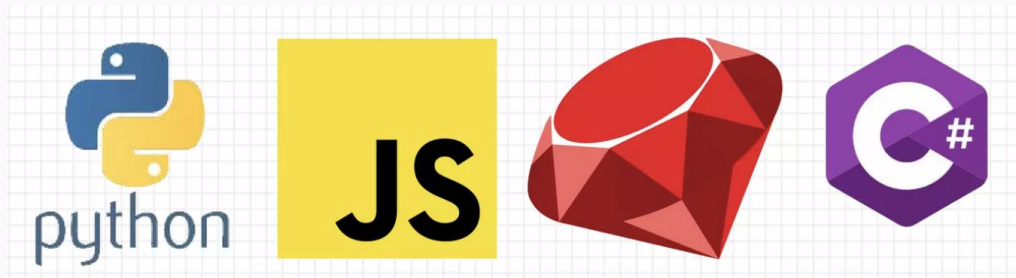
- **Cassandra:** sistema de banco de dados distribuído;
- **HBase:** sistema de banco de dados distribuído;
- **Hive:** Infraestrutura de data warehouse que transforma consultas HiveQL (Dialeto SQL) em tarefas MapReduce;
- **Pig:** Linguagem de alto nível para simplificar e agilizar a criação de tarefas de análise de dados;



Processamento de dados em tempo real

Hadoop streaming é um utilitário que vem com o Hadoop. Este utilitário permite que você crie e execute Map Reduce jobs com qualquer arquivo executável ou script conforme o map e/ou o reduce.

- Suporte a outras Linguagens de Programação;
- Ex: Python, Ruby, JavaScript, C#
- Qualquer Linguagem de Programação com stdin e stdout



Hadoop é feito para criar soluções para desenvolvedores que por acaso tem muitos dados para armazenar, ou muitos dados para analisar, ou muitas máquinas para coordenar, e que não tem tempo, habilidade, ou vontade de se tornar um especialista em sistemas distribuídos a ponto de preparar uma infraestrutura para lidar com tudo isso.

Tom White, Hadoop - The Definitive Guide



FABRICIO TORQUATO

<https://www.linkedin.com/in/fabriciotorquato/>
fabricio.torquato@facens.br

Big Data

Processamento Distribuído com Hadoop e Técnica MapReduce - Parte 1

Agenda

Hadoop e o Processamento Distribuído

- Arquitetura do Hadoop.
- Escalabilidade e tolerância a falhas.

O que é Hadoop?

- Framework Java mantido pela Fundação Apache;
- Solução open-source para trabalhar com big data;
- Criado em 2005 com apoio de grandes empresas como Yahoo !

Qual funcionalidade ?

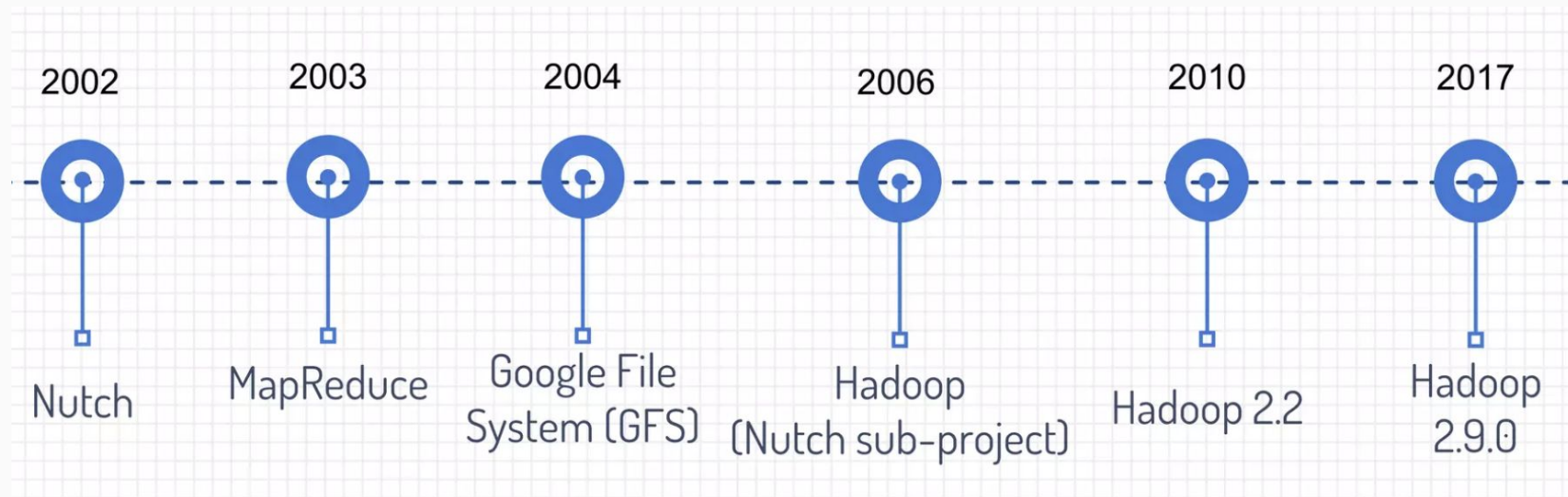
- Desenvolvimento de aplicações distribuídas;
- Processamento de grandes volumes de dados;

História

- A partir de outro projeto da Apache chamado “Nutch”.
 - Mecanismo para pesquisa web.
- Desenvolvedores encontravam dificuldades para realização de grandes processamentos em tempos eficientes;
- Doug Cutting foi o principal responsável pelo projeto;
- Yahoo ! Disponibilizou uma equipe de programadores para ajudar no desenvolvimento da ferramenta.

Hadoop e o Processamento Distribuído

História



Onde é utilizado ?

- indicado para organizações que necessitem de análise de grandes volumes de dados;
- Áreas
 - Astronomia
 - Pesquisas científicas
 - Meteorologia
 - Redes sociais
 - Buscadores
 - Mercado financeiro
 - E-commerce

Porque utilizar, quais benefícios ?

- Redução de custos;
- Boa documentação
- Comunidade ativa
- Adesão por parte de grande empresas

E como funciona ?

São 3 subprojetos dentro de um projeto maior:

- **Hadoop Common:** Utilitários comuns aos demais subprojetos;
- **Hadoop Distributed File System (HDFS):** É o sistema de armazenamento de arquivos;
- **Hadoop MapReduce:** É quem possibilita o processamento distribuído.

Pipeline de execução

1. Carregar os dados no HDFS;
2. Realizar as operações MapReduce;
3. Recuperar os resultados no HDFS;

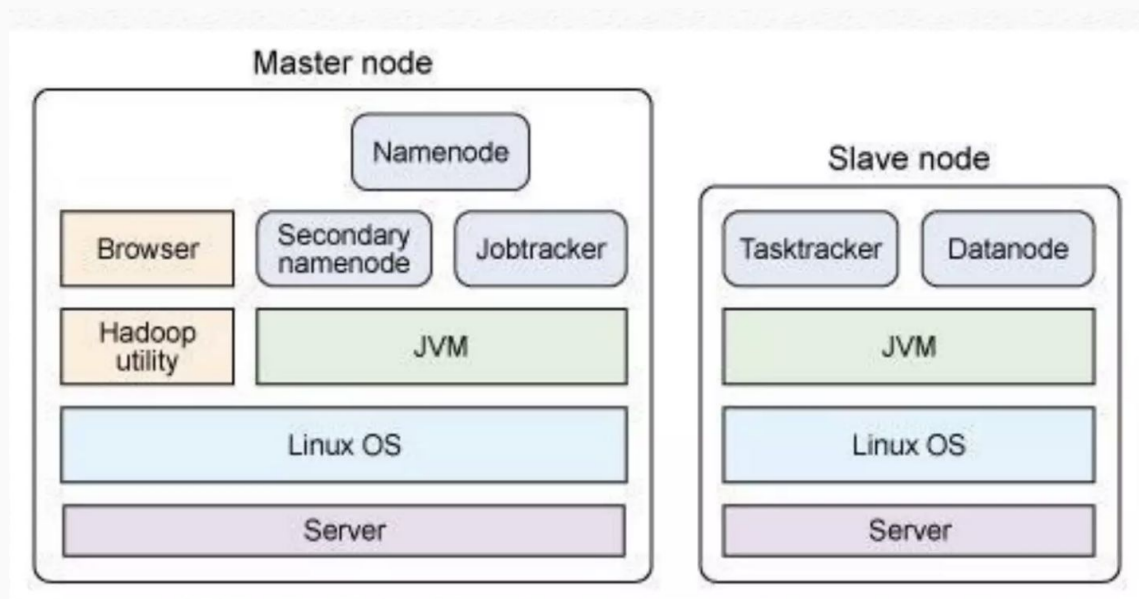
- Principal tarefa: Armazenamento de dados;
- É o sistema de arquivos do Hadoop.;
- Divide os arquivos e os réplica para diferentes máquinas do cluster;
- Desenvolvimento em Java;
- Pode ser executado em máquinas com hardwares comuns;
- Funciona sob o modelo master/workers;

Namenode

- É o master (nó principal);
- Gerencia o sistema de arquivos e o acesso a eles;
 - Conhece todos os Datanodes e onde se encontram cada pedaço de determinado arquivo;
- Representa o JobTracker, serviço que delega e monitora o status das tarefas;
 - Permite re execução de tarefas que falharam;

Datanode

- É o worker (nó participante);
- Armazena pedaços dos arquivos;
 - Um arquivo de entrada é dividido em blocos, que são replicados para diversos Datanodes;
 - Cada “pedaço” varia entre 64 MB e 128MB
- Possui um TaskTracker que executa as tarefas delegadas pelo JobTracker (do Namenode)



Tamanho padrão do bloco é 128 MB



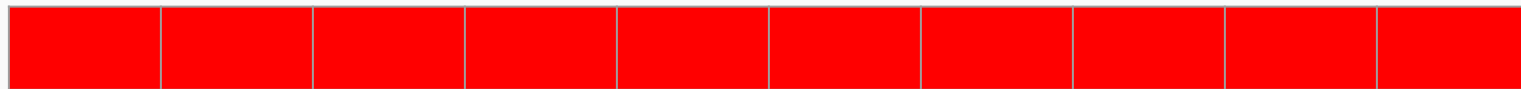
Arquivo de 64MB

Desperdício de 64MB

Se o bloco for o padrão de 128MB, um arquivo de 192 MB será dividido em dois blocos, vai ocupar um bloco inteiro e metade de outro.

Parte em vermelho é a parte ocupada

Tamanho padrão do bloco é 128 MB



Tamanho padrão do bloco é 128 MB



HDFS

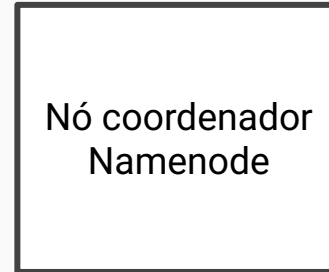
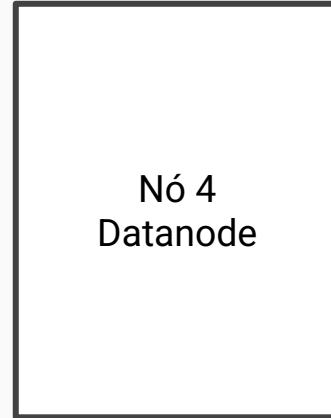
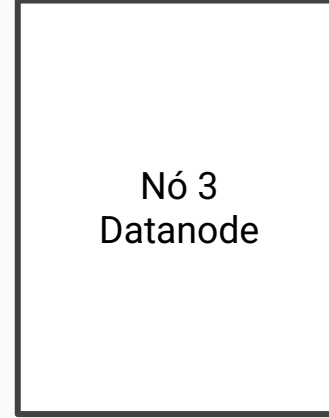
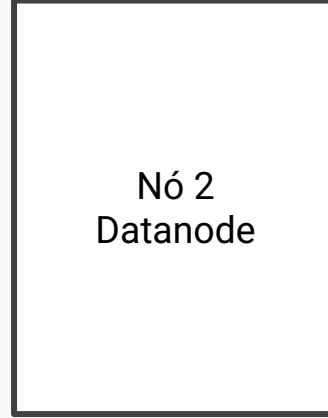
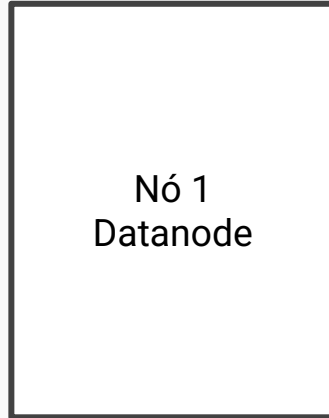
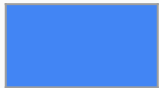
Arquivo vermelho de 248



Arquivo amarelo de 312



Arquivo azul de 64

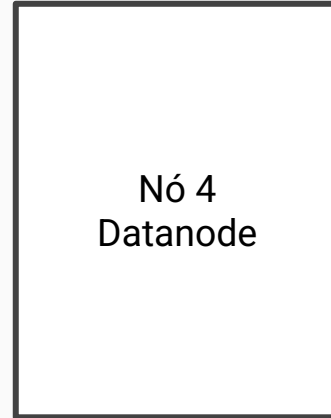
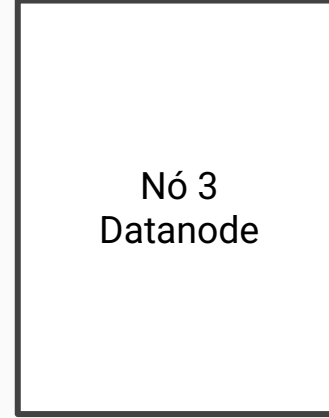
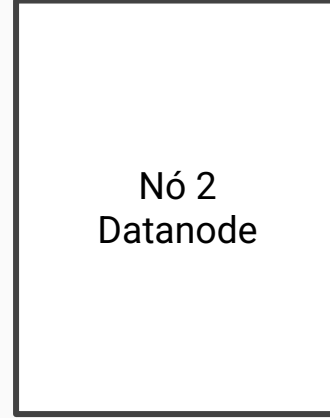
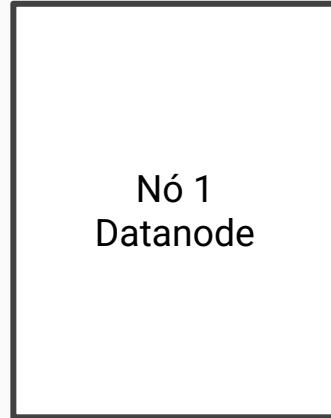


HDFS

Arquivo vermelho de 248

Divide em dois bloco de 128MB e distribui as réplicas dos blocos (padrão é 3)

1	2
1	2
1	2

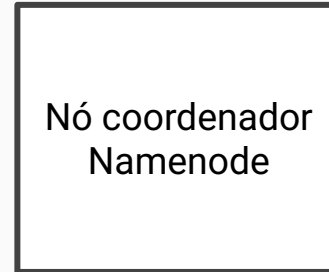


Arquivo amarelo de 312

1	2	3
1	2	3
1	2	3

Arquivo azul de 64

1
1
1



HDFS

Arquivo vermelho de 248

Arquivo amarelo de 312

Arquivo azul de 64

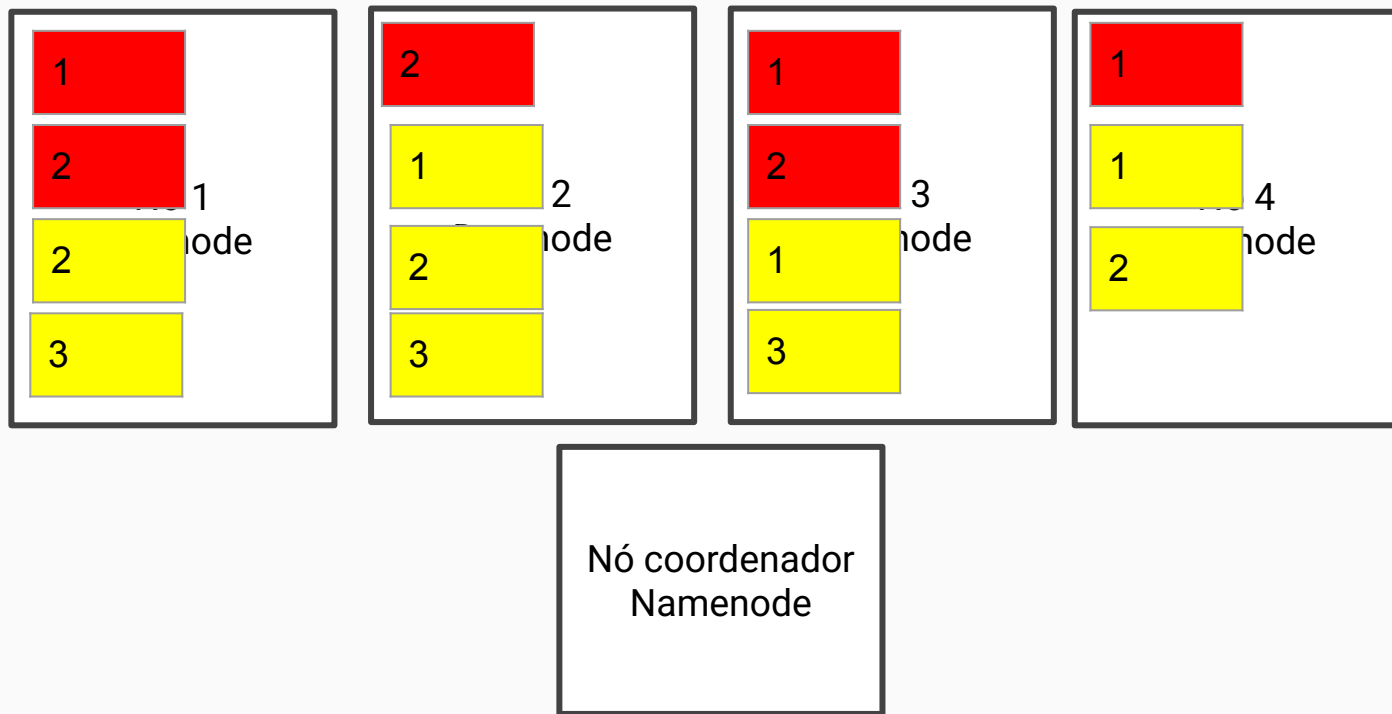


Queremos escrever o arquivo azul no cluster:

Usuário: Nó coordenador onde posso escrever?

Coordenador: Nó 2,3 e 4

Usuário -> Irá é comunicar com cada nó para escrever



HDFS

Arquivo vermelho de 248

Arquivo amarelo de 312

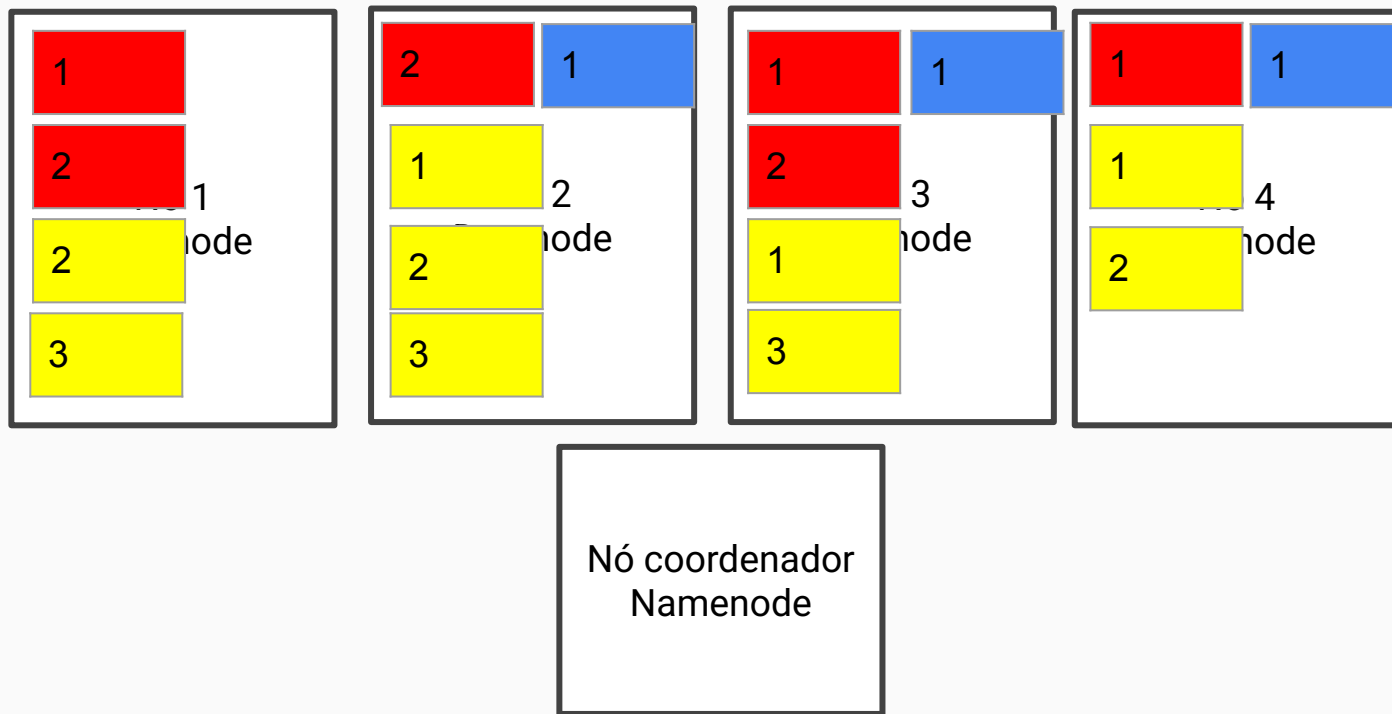
Arquivo azul de 64

Queremos ler o arquivo azul no cluster:

Usuário: Nó coordenador onde está o arquivo azul ?

Coordenador: Nó 2,3 e 4

Usuário -> Irá escolher um deles é ler



- As réplicas são não só para a segurança da integridade dos dados, mas também pela localização dos dados.
- Imaginemos que se queira fazer um Map-Reduce no arquivo azul, como ele ta nos nós 2,3 e 4. O trabalho de Map-Reduce deverá ocorrer preferencialmente nesses nós.
- No map-reduce, o processamento é levado aos dados e não os dados ao processamento.

Principais meios de executar

- API Java;
- Interface de linha de comando;

<i>Ajuda:</i>	<code>./hadoop fs -help</code>
<i>Criação de diretório:</i>	<code>./hadoop fs -mkdir my_folder</code>
<i>Listagem de arquivos:</i>	<code>./hadoop fs -ls my_folder</code>
<i>Envio de arquivo:</i>	<code>./hadoop fs -put /tmp/my_file.txt my_folder</code>
<i>Visualização de arquivo:</i>	<code>./hadoop fs -cat my_folder/my_file.txt</code>
<i>Busca de arquivo:</i>	<code>./hadoop fs -get my_folder/my_file.txt</code>
<i>Remoção de diretório:</i>	<code>./hadoop fs -rmr my_folder</code>

Meta

- Tem múltiplos clusters com Hadoop;
- Um dos maiores com 2500 núcleos de CPU e 1 PB de espaço em disco;
- Diariamente carrega 250 GB de dados para o sistema de arquivos Hadoop;
- Possui centenas de jobs do Hadoop sendo executados contra esse conjuntos de dados;
- Principal objetivo: recomendações (amigos, interesses, atividades, etc..)



FABRICIO TORQUATO

<https://www.linkedin.com/in/fabriciotorquato/>
fabricio.torquato@facens.br

Big Data

Data Lake e Esquema de Dados Não-Relacional

Agenda

Arquitetura de um Data Lake

- Camadas do Data Lake.
- Integração com outros sistemas.

Modelagem Prática de Esquemas Não-Relacionais

- MongoDB, Cassandra, e outros.
- Escolhas de design e desafios.

Estratégias Avançadas de Armazenamento

- Parquet, ORC e formatos de armazenamento colunar.
- Particionamento de dados.

O que é um Data Lake?

Um Data Lake é um repositório centralizado que permite o armazenamento qualquer quantidade dados, estruturados e não estruturados.

O que é um Data Warehouse?

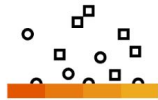
Data Warehouse (DW) é um repositório centralizado que armazena dados de diversas fontes para suportar análises e tomada de decisões. Ele é otimizado para consultas complexas e relatórios.

Diferenças entre Data Warehouse e Data Lake ?

Enquanto um Data Warehouse é estruturado e focado em dados históricos, um Data Lake é mais flexível, aceitando dados em sua forma bruta. O DW é ideal para consultas analíticas, enquanto o Data Lake suporta dados não estruturados e análises exploratórias mais amplas.

Diferenças entre Data Warehouse e Data Lake ?

Data warehouse



Raw data



Formatted and
processed data

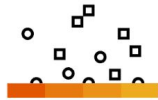


Data warehouse



Users

Data lake



Raw data



Data lake



Formatted and
processed data



Users

Diferenças entre Data Warehouse e Data Lake ?

Garrafas de água:

- Limpas
- Tratadas
- Empacotadas
- Prontas para o consumo

DATA WAREHOUSE	vs.	DATA LAKE
structured, processed	DATA	structured / semi-structured / unstructured, raw
schema-on-write	PROCESSING	schema-on-read
expensive for large data volumes	STORAGE	designed for low-cost storage
less agile, fixed configuration	AGILITY	highly agile, configure and reconfigure as needed
mature	SECURITY	maturing
business professionals	USERS	data scientists et. al.

Lago de água:

- Bruto
- Sem tratamento
- Precisa ser trabalhada para ser consumida

Para sua execução é necessária uma plataforma que contemple

- Diversas tecnologias de fontes de dados
- Processamentos de Batch's e Realtime
- Dados transacionais (se necessário)
- Robustez
- Escalabilidade

Arquitetura de um Data Lake

- On Premises (Maior Desembolso Inicial)
- Cloud com Serviço Proprietário (Existe Vendor Lock-In)
- Híbrido (Dados “Quentes” em arquitetura Local e dados “frios” na nuvem)

A Tecnologia escolhida deve lidar com os tipos de dados existentes

- Estruturados – Bancos de Dados
- Semi Estruturados – CSV, TXT, XML, JSON
- Não Estruturados

Movimento dos dados:

Os data lakes permitem a importação de qualquer tipo de dado de várias fontes no formato nativo.

Isso permite que as empresas se ajustem à quantidade de dados como necessário, sem necessidade de definir estruturas, esquemas e transformações de dados, o que resulta em economia de custos indiretos.

Armazenar e catalogar os dados com segurança:

O data lake armazena dados estruturados, semiestruturados e não estruturados vindos de várias fontes, como dados de negócios de software CRM ou ERP, dispositivos IoT, mídias sociais e até dados históricos de sistemas legados.

Também permitem capturar dados em lote e streaming e, ao mesmo tempo, aplicar governança, segurança e controle. Os dados podem ser consultados diretamente ou inseridos em um data warehouse com as ferramentas certas.

Funções analíticas e Machine Learning:

Os data lakes permitem acesso às informações baseadas em cada função para executar atividades analíticas e análises de Machine Learning sem a necessidade de transferir os dados para um banco de dados separado.

Além disso, os data lakes permitem que os dados históricos sejam combinados com os dados em tempo real para refinar o Machine Learning ou os modelos de análise preditiva e fornecer resultados novos e/ou melhores.

Data lakes modernos têm três características principais:

- Uma landing zone (zona de pouso) para os dados brutos
- Uma staging zone (zona preparatória) na qual os dados são transformados tendo em mente um objetivo analítico
- Uma data exploration zone (zona de exploração de dados) onde os dados são utilizados por aplicativos e funções analíticas e alimentam os modelos de Machine Learning

A partir do data lake, as informações são fornecidas a várias fontes, como funções analíticas e outros aplicativos de negócios, ou a ferramentas de Machine Learning para análise adicional.

Caso de uso de data lake

Os dados de vendas de longo prazo são armazenados em um data lake juntamente com os dados não estruturados, como fluxo de cliques em sites, clima, notícias e dados micro/macroeconômicos.

O armazenamento desses dados juntos e acessíveis facilita para o cientista de dados combinar as diversas fontes de informações em um modelo para prever a demanda de um produto ou linha de produtos específica.

Essas informações são usadas como entrada no sistema ERP de varejo para promover aumentos e reduções no plano de produção.

Integração com Outros Sistemas

- Alinhe o Data Lake com a Estrutura Organizacional
- Crie áreas (Zones) no Data Lake (ingest zone, transformation zone, presentation zone)
- Processos de Ingestão de Dados
- Segurança
- Linhagem de Dados
- Entender as necessidades
- Integrações serão necessárias !

Integração com Outros Sistemas

Ecosystem Hadoop



Apache Pig



Ecossistema Hadoop

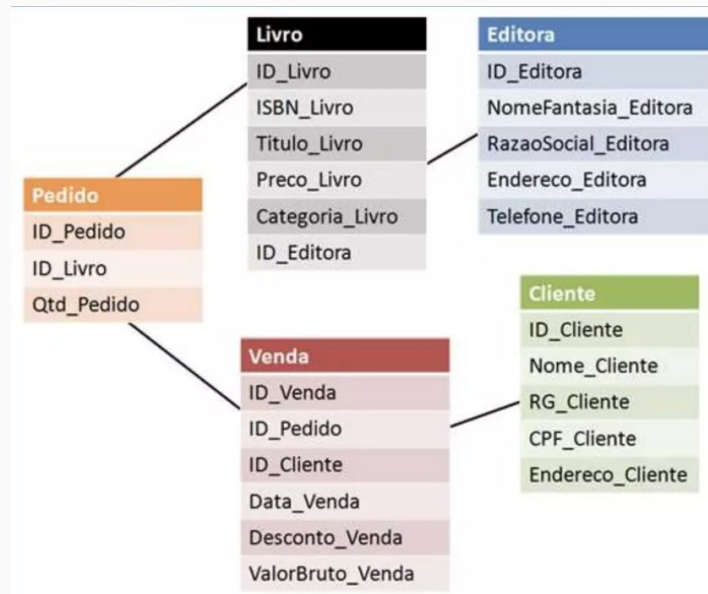
Necessidade	Tecnologia Hadoop
Gerenciamento de Metadados	Apache Atlas
Gerenciamento de Ciclo de Vida Dados	Apache Falcon
Gerenciamento de Segurança	Apache Ranger e Apache Knox
Data Science e Machine Learning	Python, Scala, Spark, etc.
Cargas de Dados	Sqoop, Hive, Spark, Kafka, Ni-Fi

Modelagem Prática de Esquemas Não-Relacionais

O conceito foi criado por **Edgar Frank Codd** em **1970**, sendo descrito no artigo "Relational Model of Data for Large Shared Data Banks".

O **modelo relacional** é um **modelo de dados**, adequado a ser o modelo subjacente de um **Sistema Gerenciador de Banco de Dados** (SGBD), que se baseia no princípio em que todos os dados estão guardados em tabelas

Historicamente ele é o sucessor do **modelo hierárquico** e do **modelo em rede**. Estas arquiteturas antigas são até hoje utilizadas em alguns data centers com alto volume de dados, onde a migração é inviabilizada pelo custo que ela demandaria;



Modelagem Prática de Esquemas Não-Relacionais

O termo **NoSQL** surgiu em -1998, a partir de uma solução de banco de dados que **não oferecia uma interface SQL**, mas esse sistema ainda era baseado na arquitetura relacional.

- Performance !!!!!

Posteriormente, o termo passou a representar soluções que promoviam uma alternativa ao Modelo Relacional, tornando se uma abreviação de **Not Only SQL** (não apenas SQL)



Modelagem Prática de Esquemas Não-Relacionais



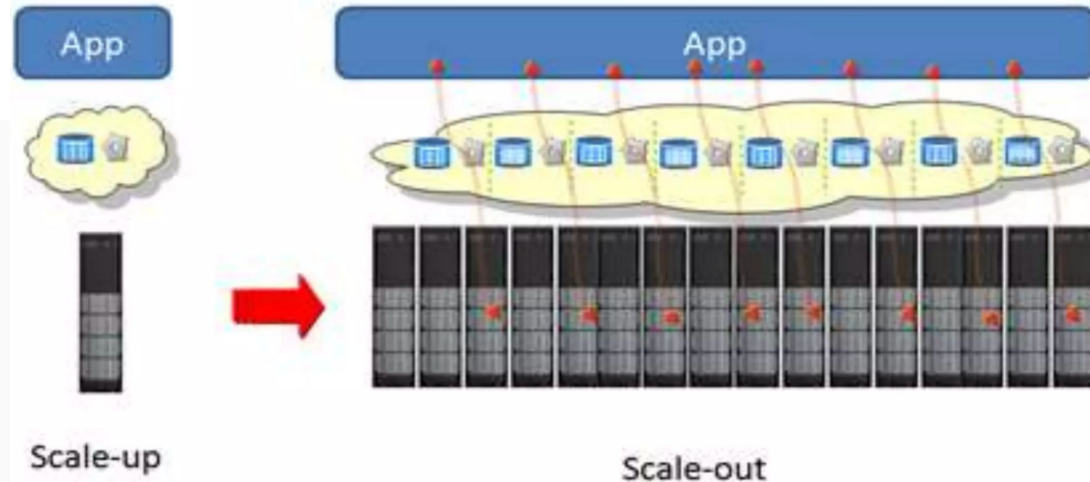
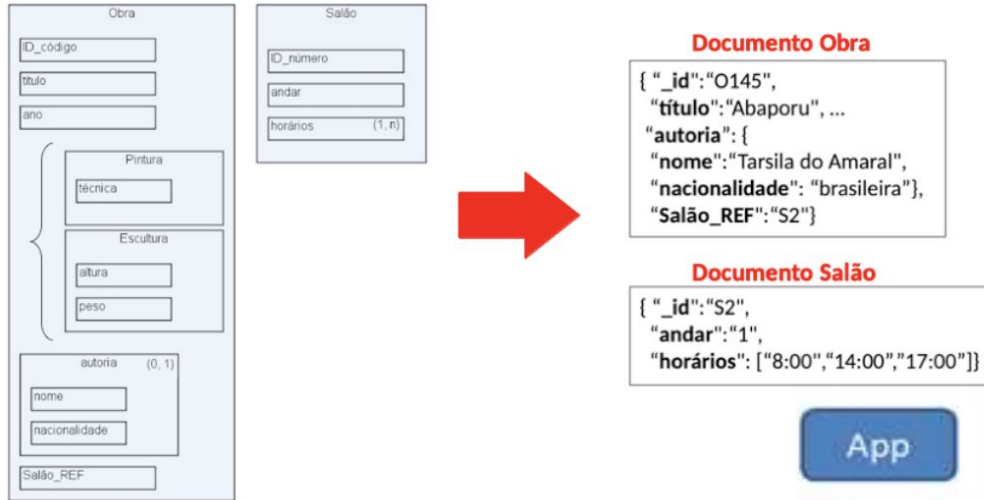
Característica em comum

- livre de esquema,
- promoverem alta disponibilidade
- maior escalabilidade

Característica em singulares (certos sistemas promovem)

- o particionamento e a replicação dos dados
- sistemas baseados em armazenamento chave-valor
- sistemas orientados a documentos
- ***sistemas orientados a coluna***
- sistemas baseados em grafos

Modelagem Prática de Esquemas Não-Relacionais



Modelagem Prática de Esquemas Não-Relacionais

O que é um banco de dados em colunas?

example database

row-name	col-1	col-2
row-1	namespace-1	value-1
row-2	namespace-2	value-2



col-2	value-1	value-2
-------	---------	---------

columnar-stores
are stored per
column

Columnar-Store

row-name	row-1	row-2	col-1	namespace-1	namespace-2	col-2	value-1	value-2
----------	-------	-------	-------	-------------	-------------	-------	---------	---------

Table

	Country	Product	Sales
Row 1	India	Chocolate	1000
Row 2	India	Ice-cream	2000
Row 3	Germany	Chocolate	4000
Row 4	US	Noodle	500

Row Store

Row 1	India	Chocolate	1000
Row 2	India	Ice-cream	2000
Row 3	Germany	Chocolate	4000
Row 4	US	Noodle	500

Column Store

Country	India	India	Germany	US
Product	Chocolate	Ice-cream	Chocolate	Noodle
Sales	1000	2000	4000	500

CSV



- CSV (Comma Separated Values)
- É um formato de arquivo simples que armazena dados em linhas, com cada linha representando uma observação.
- Os dados geralmente são separados por vírgulas, podendo haver outros tipos de separadores como Pipe (|), Ponto e Vírgula (;), Tabulação e etc. CSV é um formato de arquivo texto simples, é fácil de entender e editar manualmente.
- Não é otimizado para leitura eficiente ou compressão.
- Amplamente utilizado para exportar e importar dados de sistemas diferentes, mas não é eficiente para análises de Big Data.

Parquet



- É um formato de arquivo colunar que armazena dados em colunas separadas.
- Formato de coluna altamente eficiente e otimizado para análises de Big Data.
- Em vez de armazenar os dados linha por linha, ele os armazena coluna por coluna. Isso resulta em uma leitura mais eficiente para consultas que envolvem apenas algumas colunas, pois o sistema só precisa acessar os dados necessários.
- Compressão avançada para economia de espaço e alta velocidade de leitura.
- Suporta tipos de dados complexos. Um ponto importante é que este formato preserva a tipagem original do arquivo quando é utilizado como formato final após um processo de extração de dados.
- Pode ser lido por vários ecossistemas como Apache Spark, Hive, Impala, entre outros.

Avro



- É um formato de arquivo colunar e flexível que pode ser usado para armazenar dados estruturados e não estruturados.
- Compatível com várias linguagens de programação.
- Suporta evolução do esquema (permitindo adicionar ou modificar campos sem interromper a compatibilidade).
- Formato de arquivo binário que armazena dados em um esquema JSON.
- É usado principalmente em sistemas Hadoop.

ORC



- ORC (Optimized Row Columnar)
- É um formato de arquivo colunar otimizado para leitura eficiente e compressão de dados.
- Oferece recursos como compressão, estatísticas de coluna e suporte para tipos de dados complexos.
- Adequado para consultas analíticas em ambientes de Big Data
- Altamente compatível com o ecossistema do Apache Hadoop.

Modelagem Prática de Esquemas Não-Relacionais

Formato	Características	Uso Principal
CSV	Simples, texto, sem otimização, fácil de ler/gravar	exportação/importação de dados, pequenas tabelas
Parquet	Colunar, alta eficiência de leitura/compressão	Análise de Big Data, ecossistemas como Spark
Avro	Binário, compacto, esquema JSON	Transferência de dados em sistemas Hadoop
ORC	Colunar, eficiência de leitura/compressão	Consultas de alto desempenho com Hive/Spark

Particionamento de dados:

O particionamento dos dados é uma estratégia útil para melhorar o desempenho e a escalabilidade em projetos de Big Data.

Ao dividir os dados em partições menores, é possível distribuí-los em vários nós do cluster, permitindo um processamento paralelo mais eficiente.

Além disso, o particionamento pode ser realizado com base em critérios como o valor de um determinado campo ou a faixa de datas, o que auxilia no balanceamento de carga e na otimização das consultas.

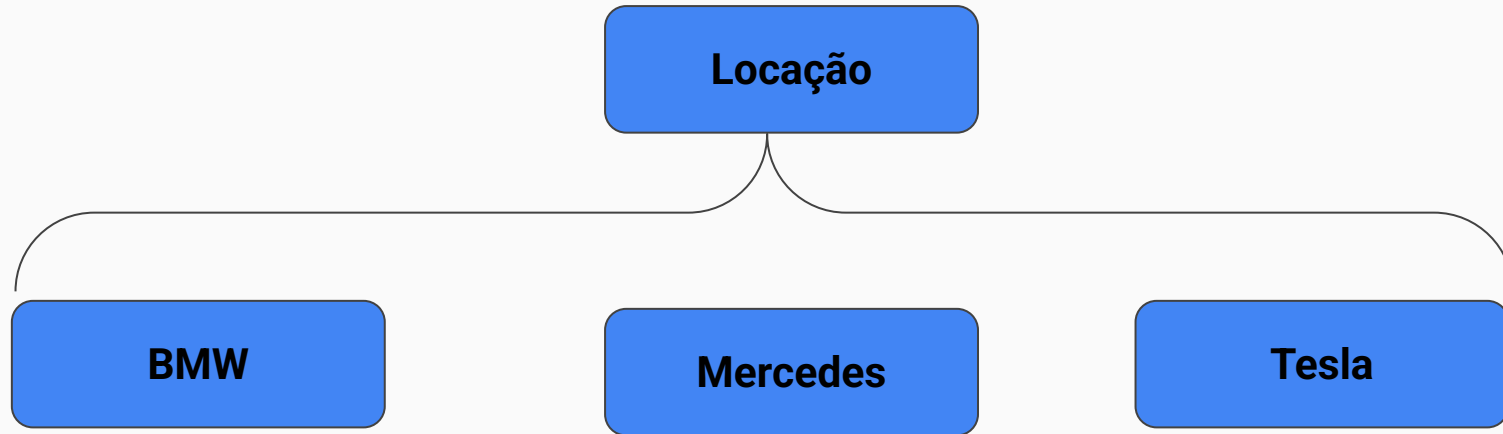
Partition

- Divide as tabelas baseado em partições lógicas e físicas
- Um partição lógica é uma pasta
 - Por exemplo, dividir locações por veículos

Objetivo:

- Otimização de consulta, já que a partição fica fisicamente separada
 - Se fizermos uma consulta na tabela vendas onde ***where veiculo = 'Tesla'***, a consulta vai executar de forma muito mais otimizada!
- Não há benefícios em simples consultas (ex, select * from vendas)

Partition



Bucketing

- Partições: variam conforme os dados.
 - Potencial problema: milhares ou milhões de partições!
- Bucketing: número Fixo de partições, não muda conforme os dados
 - Divide fisicamente de forma balanceada em partições
 - Se existirem 300 partições e 50 dados diferentes, 250 ficarão vazios!
 - Ótimo para tabelas que operam em Joins

Partition:

- Baixa Cardinalidade
- Um atributo ou mais

Bucketing:

- Cardinalidade alta e variável
- Apenas um atributo



FABRICIO TORQUATO

<https://www.linkedin.com/in/fabriciotorquato/>
fabricio.torquato@facens.br

Big Data

Introdução a Big Data e seus Conceitos



Agenda

Definição de Big Data

- Os 3Vs e além: Volume, Velocidade, Variedade e Veracidade.
- Conceitos fundamentais: Batch vs. Streaming.

Casos de Uso em Diferentes Setores

- Saúde: Análise de dados médicos.
- Finanças: Detecção de fraudes.
- Agricultura: Otimização da produção.

Desafios e Vantagens do Big Data

- Desafios técnicos: Armazenamento, Processamento, etc.
- Vantagens estratégicas para negócios.

Tendências Atuais em Big Data

- IA e Aprendizado de Máquina em Big Data.
- Edge Computing e seu impacto em grandes volumes de dados.

O que é Big Data?



O que é Big Data?

Nossa sociedade está deixando para trás uma pegada digital.

As pessoas estão vivendo em linha e todos estamos expressando nossas atitudes, gostos, desgostos, opiniões e perspectivas.



O que é Big Data?

Estamos gerando enormes quantidades de dados. Empresas estão capturando trilhões de bytes de informação sobre os seus clientes, fornecedores e suas operações.



O que é Big Data?

Indivíduos com smartphones e sites de redes sociais continuarão a impulsionar o crescimento de dados exponencialmente.

O que é Big Data?

90% dos dados no mundo de hoje foram criados nos últimos anos.



O que é Big Data?



O que é Big Data?

A maioria das empresas não sabem o que fazer com todos os dados que eles já tem

Data-driven design

O que é Big Data?

“Big Data é a capacidade de uma empresa para armazenar, processar e acessar todos os dados de que necessita para funcionar, tomar decisões, reduzir riscos, e servir os clientes” - Forrester

“Big Data, em geral é definido como alto volume, velocidade e variedade ativos de informação que exigem formas eficazes e inovadoras de custos de processamento de informações para uma melhor percepção e tomada de decisão” - Gartner

“Big data são os dados caracterizados por três atributos: volume, variedade e velocidade” - IBM

“Big data são os dados caracterizados por quatro atributos principais: volume, variedade, velocidade e valor” - Oracle

O que é Big Data?



Volume:

Se não houver um volume mínimo de dados como poderemos inferir estatisticamente se as tendências detectadas estão corretas?

Velocidade:

A necessidade de obtermos dados, inclusive em tempo real, sua gravação, alteração ou substituição deve ser feito em tempo hábil para que gere ganhos reais.

Variedade:

É outro aspecto importante. Os dados podem e devem ser tratados conforme sua origem – estruturados e não estruturados. Os dados estruturados são aqueles cuja origem é oriunda de bancos de dados relacionais ou não (Oracle, SQLSERVER, etc.) e de dados esparsos como, por exemplo, vídeos, e-mails, documentos, imagens, etc.

Veracidade:

De nada obtermos um alto volume de dados se os mesmos não são confiáveis.

Valor:

Informação não é só poder, informação também é patrimônio. A combinação “volume + velocidade + variedade + veracidade”, além de todo e qualquer outro aspecto que caracteriza uma solução de Big Data, se mostrará inviável se o resultado não trazer benefícios significativos e que compensam o investimento.

O que é Big Data?

Portanto, Big Data = volume + variedade + velocidade +
veracidade, tudo agregando + valor

Batch vs. Streaming

Batch Processing:

Processa dados em lotes, ideal para grandes volumes, com intervalos definidos.

Streaming Processing:

Processa dados em tempo real, adequado para análise contínua e resposta imediata.

Casos de Uso em Diferentes Setores

Um exemplo interessante do uso de Big Data na pesquisa de medicamentos é o cruzamento de informações entre usuários de diversas drogas. Suponhamos que uma pessoa que sofra de pressão alta tome determinado medicamento.

Ela é alertada de efeitos colaterais como palpitações. Um outro indivíduo está se medicando com antidepressivos e também é alertado de efeitos colaterais.

Mas, se uma pessoa está tomando os dois remédios ao mesmo tempo?

Qual o efeito colateral resultante?

Administração Pública:

Soluções inovadoras para transporte e trânsito

Exemplo:

Em Singapura, um sistema pode prever velocidades no tráfego com precisão de 90%.

Segurança Pública:

Exemplo:

Dados coletados de diversas fontes, que vão de câmeras nas ruas a comentários e posts publicados em mídias sociais, as agências de inteligência e de segurança pública podem detectar e se antecipar a atividades ilícitas, evitando que ocorram.

- **Processamento eficiente:**
 - Computação e Análise dos Dados
 - Velocidade;
 - Utilização de índice não é adequada para dados complexos;
 - Construir e adaptar algoritmos para trabalhar de forma paralela;
 - Aproveitar a infraestrutura em nuvem;
 - Desempenho
 - Congestionamento da rede

- **Segurança**

- Permitir análise de dados sem informações confidenciais;
- Tecnologias atuais para proteção da privacidade consideram dados estáticos;

- **Escalabilidade**

- Construir sistemas para processar um volume crescente de dados;
- Volume está aumentando em uma velocidade maior do que os recursos computacionais;
- Crowd-sourcing

- **Análise para Big Data**

- Alta dimensionalidade dos dados;
- Acúmulo de outliers;
- Correlação incorreta dos dados;
- Alto custo computacional;
- Necessidade de algoritmos complexos;

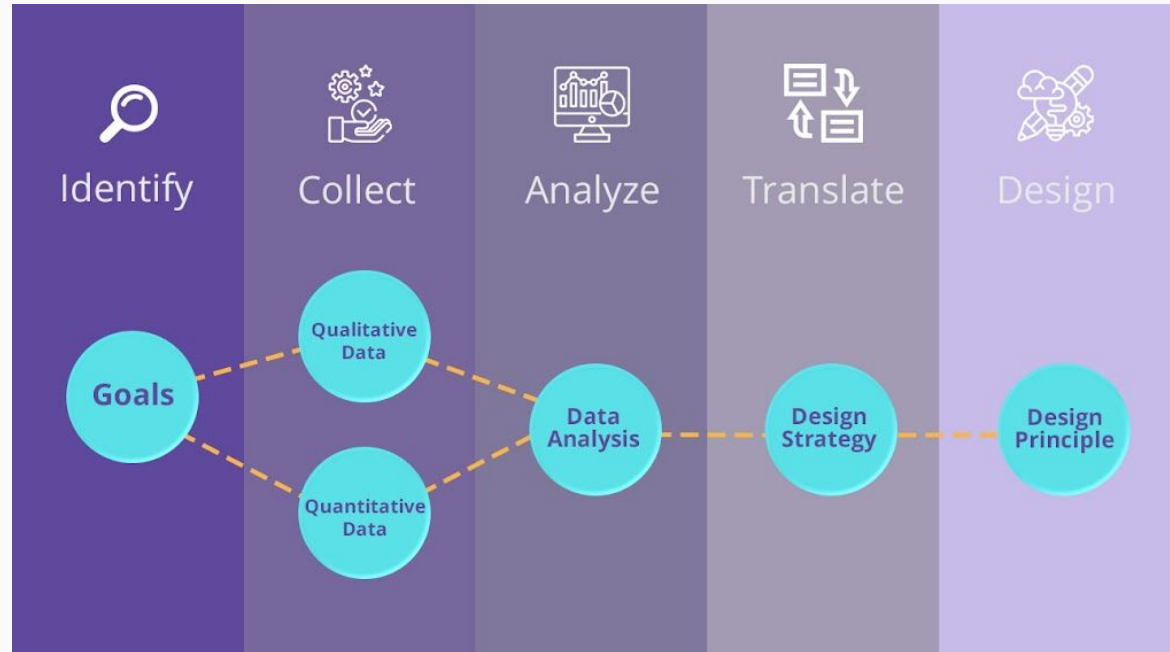
- **Heterogeneidade**

- Múltiplas fontes, diferentes pontos de tempo e diferentes tecnologias;
- Necessidade de procedimentos robusto e adaptáveis;

- **Armazenamento de Big Data e Cloud**
 - Necessidade de gerenciar grandes infraestruturas;
 - Hadoop é complexo e sem suporte gerencial;
 - Mais de 150 variáveis a serem configuradas;

- **Armazenamento de grandes volumes de dados:**
 - Tecnologias atuais não são adequadas para Big Data;
 - Capacidade de armazenamento cresce mais lentamente do que a quantidade de dados;
 - Algoritmos não são eficientes para trabalhar com heterogeneidade de dados;
 - Incompletude dos dados;
- **Computação em Nuvem - Desvantagens**
 - Privacidade
 - Dados armazenados no provedor
 - Dependência de provedor em nuvem

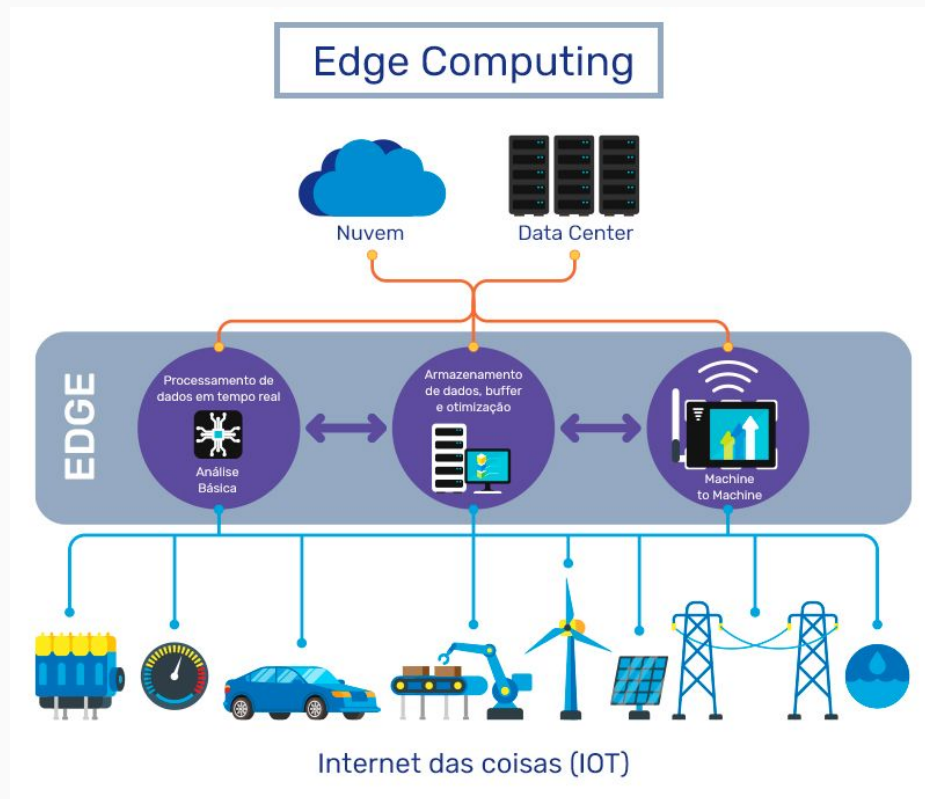
Transformando Dados em Estratégias



IA e Aprendizado de Máquina em Big Data



Edge Computing e seu impacto





FABRICIO TORQUATO

<https://www.linkedin.com/in/fabriciotorquato/>
fabricio.torquato@facens.br